

Universidad de la Sabana, Facultad de ingeniería

Robo Cup Logistics League: Búsqueda e identificación de códigos QR

Juan Felipe Garzón Romero

Nicolás Hernández Leguizamón

Santiago Esteban Pinzón Botero

Jorge Alberto Castellanos Rivillas

Mauricio Alejandro sierra Cetina

Chía, 2022-1

Contenido

Introducción:	4
Alcance y metodología:.....	5
Objetivos:	8
Objetivo general:	8
Objetivos específicos:	8
Contexto:	9
RoboCup Logistics League y reglamentación.....	9
Área de competición y máquinas:	9
Etapa 1: Fase de exploración:.....	10
Etapa 2: Fase de producción.....	11
Sanciones:	13
Sistema móvil Robotino 3 FESTO:.....	13
Odometría:.....	15
Desarrollo Proyecto (Fase 1 Robocup):	17
Elección lenguaje de programación	17
Descarga de librerías necesarias.....	18
Robotino API C++	18
OpenCV.....	18
ArUco.....	18
Uso de instrumentos del Robotino: Cámara.....	19
Recorte de imágenes.....	19
Detección de colores	20
Detección marcadores ArUco	23
Laser rangefinder.....	25
Sensores Infrarrojos	26
Disposición del área de trabajo	27
Algoritmo de búsqueda	30
Momento 1. Encontrar cuadrante de las cajas	30
Momento 2. Ir al cuadrante con caja más cercano	35
Momento 3. Búsqueda marcador ArUco.....	38
Búsqueda de cajas no identificadas en el primer paso.	40
Conclusiones:	42
Referencias:	43

Anexos:	44
Anexo A: Clases propias	44
Anexo B: Clases API Robotino.....	45
Anexo C: Objetos	45
Anexo D: Funciones.....	46
Anexo E: Instalación dependencias y librerías para correr el programa en C++	52
Anexo F: Diagrama de Gantt	62

Introducción:

El futuro de la producción industrial es dejar de utilizar los mismos sistemas mecánicos para crear y/o adaptar sistemas más inteligentes y flexibles, así como autónomos, aumentando la productividad de la industria a un nivel mayor que el que existe actualmente. Para esto, compañías pioneras en tecnología buscan el talento para poder desarrollar este tipo de maquinaria, objetivo que tiene la competencia Robocup, donde se reflejan diferentes aplicaciones haciendo uso de la robótica; está claro que la escala de los escenarios y robots es reducida comparado con lo que sería necesario para la industria real, como ejemplo, están los robots que provee FESTO (Robotino). Estos sistemas móviles tienen como finalidad educar, formar y ser el medio de investigaciones para ayudar a crear soluciones a problemas dentro y fuera de la industria.

Este documento abarca el estudio y desarrollo del proyecto que se llevó a cabo el semestre 2022-1 para la asignatura diseño mecánico en énfasis del pensum de asignaturas de Ingeniería Mecánica. Este proyecto tiene como finalidad sentar las bases iniciales de código para la participación en la “RoboCup Logistics League” a estudiantes que estén interesados en participar. Este proyecto permitió al grupo de trabajo aprender sobre el mundo de la robótica y la programación, pasando de conocer muy poco sobre ciertos lenguajes de programación (C++), a desarrollar un algoritmo completamente funcional para la competencia. Este texto es fundamental para poder guiar a los futuros desarrolladores del proyecto para que aprovechen todos los recursos al máximo y tengan una guía para continuar con el trabajo, ya que lo realizado hasta el momento es únicamente la Fase 1 de la competencia (identificación del terreno). Este algoritmo debe realizarse teniendo en cuenta parámetros básicos para la participación en dicha competición y debe estar claro que los objetivos presentados en este documento deben cumplirse de igual forma.

En este documento se dará a conocer el algoritmo con el cual el Robotino 3 es capaz de cumplir con la búsqueda e identificación de códigos AR, de igual forma se plasma el desarrollo y función de los diferentes apartados del código, lo cual comprende en su totalidad el desarrollo del proyecto, y se resaltan las conclusiones a las que se llegó.

Alcance y metodología:

El enunciado concreto de lo que se debe llevar a cabo es tomar un ambiente cerrado y delimitarlo para que en este el Robotino 3 sea capaz de buscar e identificar códigos AR (propios de las MPS), identificando el/los códigos, posicionándose a una distancia adecuada, guardando la posición y continuar con la búsqueda del siguiente código (Etapa 1 de la competencia). El algoritmo debe hacer que el Robotino 3 cumpla con los siguientes aspectos: Velocidad de detección y movimiento hacia la ubicación de los AR, posicionamiento y ruta para llegar a las estaciones donde se ubican los códigos AR que identifican a las ya mencionadas, finalizado, la detección de las estaciones donde están los códigos no debe repetir la estación en caso tal de que sea detectada.

Para llevar a cabo el desarrollo del proyecto se definieron aspectos fundamentales para cumplir con el propósito del proyecto. En primer lugar, en cuanto al software, el lenguaje de programación que se concretó para trabajar en el desarrollo del algoritmo fue C++, haciendo uso de Microsoft Visual Studio, el cual es un entorno de desarrollo integrado para Windows el cual facilitó el trabajo debido a su facilidad de instalación. Se utilizó un paquete de la interfaz de programación denominada API (propia del sistema operativo de Robotino) para la creación de programas en C++ y generar el vínculo para la programación de los componentes del Robotino. Se utilizó una biblioteca de visión artificial denominada OpenCV, software libre de código abierto, que permite trabajar con todo lo relacionado al procesamiento de imágenes capturadas por una cámara, el cual es programado en C++. Estos son fundamentales para hacer que el algoritmo desarrollado para el Robotino funcione. También se utilizaron otros programas complementarios que se explica a detalle su instalación y la de los demás programas en la sección de *Desarrollo*.

En cuanto al hardware y los recursos externos como el espacio donde se posiciona el Robotino y las estaciones con los códigos AR, se utilizó la cámara del Robotino 3 para poder realizar el mapeo, identificar los AR, identificar las estaciones, calcular las distancias, entre otras cosas que se explican en el apartado *Desarrollo*. Se utilizaron computadores portátiles para trabajar en el código, se recomienda el uso de computadores que tengan una buena capacidad de almacenamiento y que tengan un buen rendimiento (características propias de los equipos). Se adecuó el espacio para permitir que el Robotino se desplazara sin inconvenientes (espacio localizado en el segundo piso del edificio FabLab de la Universidad de La Sabana, Chía, Cundinamarca, este cuenta con un espacio de 2.8 m de ancho x 7 m de largo. Al no poder acceder a las MPS se fabricaron las estaciones a partir de cajas de 20 cm de ancho x 30 cm de largo x 50 cm de alto, pintadas de color naranja para facilitar su reconocimiento por cámara, a estas cajas se les pegaron los códigos AR (las estaciones fabricadas se encuentran en el laboratorio de automatización y robótica del FabLab), se fabricaron 3 estaciones, debido a que posicionar un número mayor de estaciones no era pertinente para el espacio propuesto, ya que no se cumpliría con las reglas de posicionamiento de las estaciones dadas por la Robocup Logistics League (reglas en el apartado *Contexto*).

La manera en la que se organizó el grupo de trabajo para el desarrollo del proyecto está ligada a la planificación de cada tarea mediante un diagrama de Gantt (Ver anexo F), esto fue esencial para ejecutar, adaptar y evacuar las tareas que requerían menor tiempo (adecuación del espacio, instalación software, paquetes, entre otros), posterior a esto, se empezó con la búsqueda de información relevante para romper con la curva de aprendizaje y finalmente desarrollar los 3 aspectos necesarios para el cumplimiento de los objetivos del proyecto, presentados en la sección *Objetivos*.

A lo largo del proyecto se presentaron varios inconvenientes, Se recomienda al grupo de estudiantes que continúe con el proyecto, que apliquen los consejos y que tengan en cuenta las recomendaciones para que no tengan problemas en la continuación del proyecto. A continuación, algunas limitantes:

- Cuando se inició el proyecto se intentaron utilizar diferentes herramientas computacionales para controlar el Robotino 3, Robot Operating System (ROS) es un framework para el desarrollo de software para robots, el cual puede presentar varios problemas en la instalación de este y de algunos complementos, adicional a esto la información que se encuentra en la red sobre ROS para Windows es bastante escasa. Sin embargo, si se tiene el sistema operativo Linux es posible que se encuentra una cantidad mayor de información que lo que se encuentra para Windows, por lo que no fue posible implementarlo.
- Se utilizaron diferentes lenguajes y herramientas para programar, como Python, Java, Matlab, LabVIEW y el propio Robotino View (este último se descartó debido a que la RoboCup no acepta algoritmos realizados en este programa). Si bien es cierto que se podría realizar un algoritmo en algunas de estas otras herramientas, la información que se encuentra en la red es escasa, además, en algunos casos no existen paquetes que permitan continuar con el desarrollo del algoritmo, como fue el caso al momento de usar LabVIEW.
- En cuanto al uso de otros lenguajes de programación es importante aclarar que Matlab para uso de herramientas de simulación virtual como lo es Robotino SimDemo no está soportado para realizar tal acción, por lo que se recomienda utilizarlo si la disponibilidad y tiempo es suficiente para trabajar de manera presencial.
- En cuanto al espacio donde se realizaban las pruebas, de ser posible buscar un lugar que tenga las características propias al espacio donde se realiza la RoboCup Logistic League buscarlo, ya que debido a que el aula donde se encuentra almacenado el Robotino es bastante pequeño y la zona donde se encuentra actualmente, es bastante transitada por los demás estudiantes. Además, tuvo que ser adecuado ya que no se permitió utilizar un espacio mayor a 3 m x 7 m, debido a lo que se comentó anteriormente.
- La zona actual de pruebas presenta una iluminación aceptable. Sin embargo, sería pertinente buscar una manera de iluminar mejor la zona para que la iluminación sea pareja en todo el espacio (en caso tal que no se vaya a trasladar la zona de las pruebas para el Robotino 3).

- Los computadores son fundamentales para el desarrollo del proyecto, por lo que se necesita de un equipo que tenga un buen espacio de almacenamiento y si es posible que sea rápido (Buen procesador y memoria RAM), ya que, si el equipo se ralentiza, la conexión entre el Robotino y este se puede caer, la compilación del código puede ser bastante demorada, la ejecución del algoritmo puede ser intermitente haciendo que el proceso se demore más de lo normal.

Objetivos:

Objetivo general:

Validar que el robot móvil para investigación y formación de FESTO (Robotino 3), sea capaz de ubicar e identificar códigos QR en un ambiente delimitado, con el fin de determinar y almacenar sus posiciones y orientaciones.

Objetivos específicos:

- Implementar el lenguaje de programación y las herramientas computacionales en el Robotino 3 para que le permitan ser rápido y eficiente en la búsqueda y ubicación de los códigos QR.
- Diseñar un código capaz de capturar imágenes utilizando la cámara del Robotino, y realizar el procesamiento de imagen adecuado, para identificar correctamente los códigos QR.
- Exponer en la ventana de comandos las posiciones de los códigos QR que representan la ubicación y orientación de una MPS, con sus respectivas coordenadas en "x" y "y".

Contexto:

RoboCup Logistics League y reglamentación

La RoboCup Logistics League (RCLL) es una subcategoría de la competencia RoboCup Industrial League, que busca promover la investigación y el desarrollo de los retos a nivel logístico de los futuros procesos de producción al interior de la industria haciendo uso de equipos coordinados de robots móviles autónomos; por lo que esta competencia tiene como objeto, que un equipo de robots realice el ensamble de productos, utilizando una serie de máquinas de producción. Estos equipos se asemejan a diferentes etapas de maquinaria utilizada en el área de producción (Búsqueda de materia prima, ensamble de piezas, entrega de componentes y/o producto final), los sistemas de producción MPS de FESTO tienen como propósito emular estas actividades. Para concatenar las operaciones, es fundamental el uso de robots que permitan trasladar el producto previamente seleccionado, armado o clasificado a la siguiente estación por la que debe pasar para adquirir valor y lograr ser el producto que se solicita para que salga al mercado; FESTO permite el uso de la plataforma robótica estandarizada Robotino la cual puede desplazarse libremente con sensores, sistemas de transmisión, entre otros.

El área dentro de la competición cuenta con el espacio para que dos equipos compartan un piso de fábrica, cada equipo cuenta con hasta tres robots autónomos los cuales comparten campo con siete máquinas de producción. Estos robots son cooperativos, es decir, que necesitan comunicarse e interactuar de forma física con las demás máquinas. Para poder interactuar con las estaciones, los robots deben ser capaces de identificar con que estación van a interactuar y realizar el proceso que tenga dicha estación, por ejemplo, extraer la materia prima de una máquina dispensadora. En la competición en la RoboCup Logistics League es indispensable el uso del sistema Robotino 3 junto a las estaciones de FESTO, al competir se deben tener presentes ciertos parámetros para que se haga un buen ejercicio a la hora de realizar el algoritmo de las estaciones fijas y móviles. A continuación, los parámetros de libre elección y las limitantes:

Área de competición y máquinas:

El área donde se lleva a cabo la competencia es de 14 m x 8 m, con un total de 106 cuadrantes de 1 m x 1m, con 14 estaciones distribuidas de manera aleatoria. El 70 % de esta zona está rodeada por paredes de 50 cm de altura, la ubicación de las estaciones en los cuadrantes se ven sujetas a una serie de posicionamientos, es decir, las estaciones pueden tener un ángulo de rotación de 0°, 90°, 180° o 270° con respecto a una horizontal del cuadrante en donde estén ubicadas (o en caso de estar ubicadas de manera diagonal, ángulos de rotación de 45°, 135°, 225° o 315°), teniendo esto en cuenta, también se habilitan o restringen las entradas por donde puede ingresar el Robotino para identificar la estación que tiene en frente, tal y como se muestra a continuación:

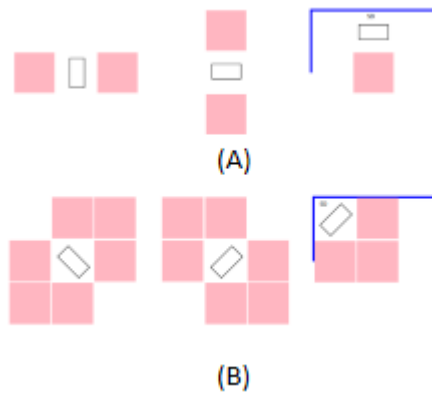


Figura 1. (A) Rotaciones 0°,90°,180°,270° (B) Rotaciones 45°,135°,225°,315° con zonas restringidas [1]

Las zonas denotadas en rosado hacen referencia a los cuadrantes donde no pueden ser ubicadas otras estaciones de procesos. Hay que tener en cuenta que las estaciones o MPS, tienen un etiquetado el cual facilita la distinción entre ellas, este etiquetado se realiza mediante códigos QR / AR los cuales tienen un ID específico para cada estación de producción o almacenamiento, un ejemplo de cómo deben ubicarse los códigos en las MPS es como se muestra a continuación:



Figura 2. Estación Base con su respectivo código de identificación [1]

Etapas 1: Fase de exploración:

El Robotino de cada equipo tiene tres minutos para recorrer el espacio y realizar un informe donde se indiquen los siguientes apartados de las MPS: Posición del cuadrante donde se ubica la MPS, la orientación de la estación (Código QR/AR) y el tipo de máquina (ID). La fase de exploración termina cuando los tres minutos hayan pasado o inmediatamente ambos equipos tengan la información solicitada sobre las MPS. En esta fase los puntos que puede obtener un equipo se basan en lo establecido en la siguiente tabla:

Informe	Descripción	Puntos
Zona/Orientación	Se determina correctamente una zona de máquinas y la rotación de su equipo	+2
Tipo	Solo zona de máquinas reportada	+1
	La zona de la maquina se informó correctamente, pero la orientación es incorrecta	0
	La zona de la máquina informada incorrectamente (orientación irrelevante)	-1
Ronda total	En total son 14 puntos si se realiza todo de manera correcta con las 7 MPS	De 0 a 14

Tabla 1. Puntuación fase de exploración

Etapas 2: Fase de producción

El Robotino tiene 17 minutos una vez finalizada la etapa de exploración, dependiendo el objeto que deba ser transportado una estación u otra. La diferencia entre objetos depende directamente de la composición de colores de estos y la cantidad de elementos apilados, esta composición de anillos de colores se representa con C0, C1, C2 y C3, tal y como se muestra continuación:

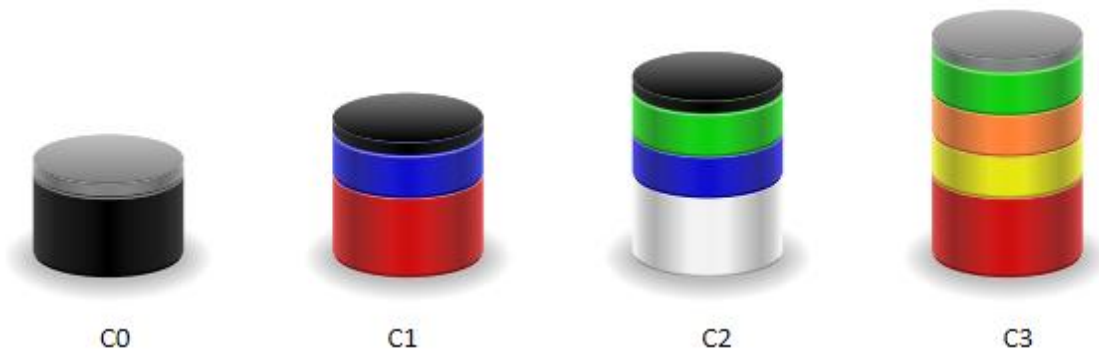


Figura 3. Productos de estaciones [1]

Las cadenas de producción de los cuatro productos son establecidas por el comité técnico de la competencia, un ejemplo puede ser el siguiente:

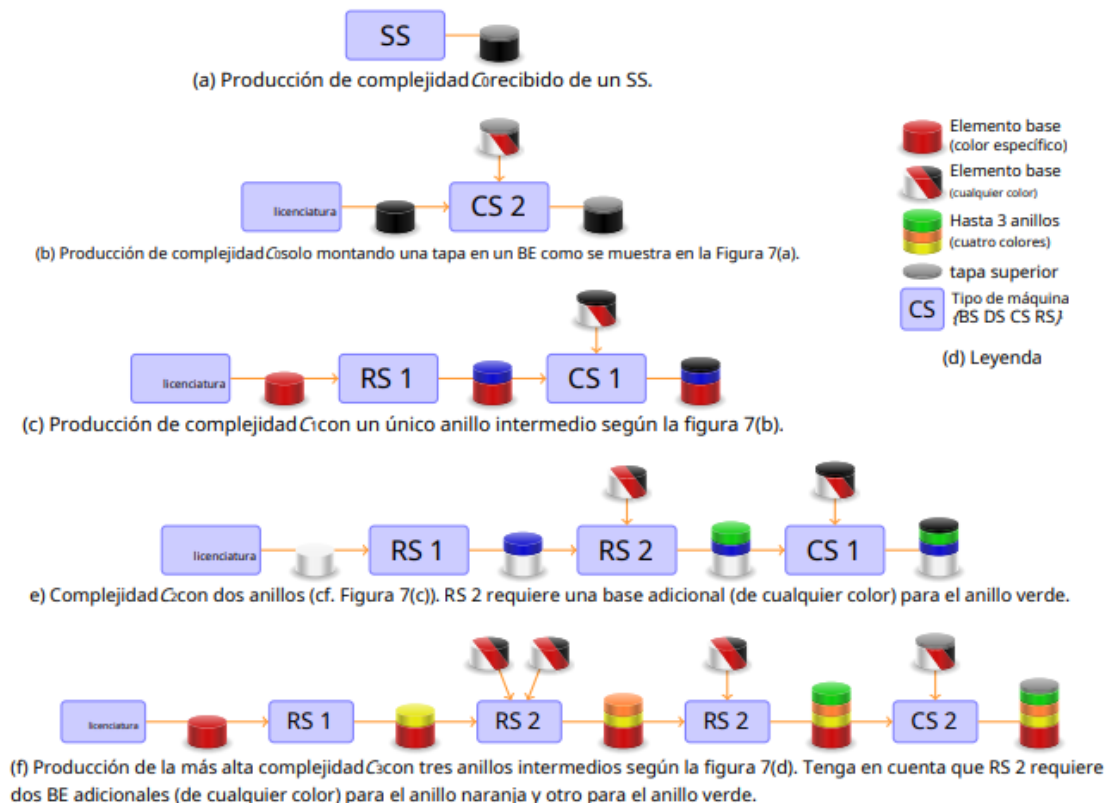


Figura 4. Procesos para cada producto [1]

Por lo que en esta fase los puntos que pueden obtener los equipos se basan en lo establecido en la siguiente tabla:

Tarea/subtarea	Fase de producción	Puntos
Base adicional	Alimentar una base adicional en la estación de anillos	+2
Terminar paso C0	Terminar la orden de trabajo para un color que no requiere base adicional	+5
Terminar paso C1	Terminar la orden de trabajo para un color que requiere una base adicional	+10
Terminar paso C2	Terminar la orden de trabajo para un color que requiere dos bases adicionales	+20
Terminar C1 Acabado	Montar el último anillo de un producto C1 y/o un producto C2	+10
Pre-tapado C2 Acabado		+40
Pre-tapado C3 tapa de montaje	Montar el último anillo de un producto C3	+80
Recuperar tapa	Almacenar una tapa en la estación de tapas	+2
Recuperar de SS	Se solicita una pieza de trabajo de la estación de almacenamiento y está listo para su obtención	-10
Entrega	Entregar una de las variantes del producto final en la zona de carga	+20
Entrega tardía	Un pedido entregado después de los 10 segundos posteriores a la orden, a medida que aumenta el tiempo tardío, menor será la puntuación.	+5
Entrega incorrecta	Un pedido entregado fuera del intervalo de tiempo designado	+1
Entrega falsa	Entregar un producto incompleto	0
Obstrucción	Entregar una pieza en estaciones de otro equipo	-20
Ronda total	Se pueden dar un mínimo de 0 puntos	0 min

Tabla 2. Puntuación fase de producción

Sanciones:

En cuanto a las sanciones que pueden ser aplicadas a un equipo se encuentran:

- *Movimiento prematuro:* El Robotino puede moverse hasta que se anuncie el inicio de la competición, si se incumple esto el robot deberá estar inmóvil durante 2 minutos una vez empezada la competencia.
- *Daños en el equipo de la fábrica simulada:* Esto será sancionado como una infracción menor de la regla.
- *No presentarse a la competencia:* Eliminación de la competencia.
- *Intervención manual:* Si no es autorizado el manipular al robot, será sancionado como una infracción mayor de la regla.
- *Romper una regla menor:* Se descontará el 25% de los puntos anotados por el equipo en el momento de la infracción (mínimo un punto).
- *Romper una regla mayor:* Se descontará el 50% de los puntos anotados por el equipo en el momento de la infracción (mínimo cinco puntos).
- *Discutir con el árbitro:* Primero se realiza un llamado de atención, si se vuelve a presentar se penalizará inmovilizando el Robotino durante el tiempo que considere el árbitro.
- *Incumplimiento en las normas de conducta:* Se penalizará inmovilizando el Robotino durante el tiempo que considere el árbitro, o en caso tal se expulsará al equipo de la competencia.

Sistema móvil Robotino 3 FESTO:

Es una plataforma con interfaces eléctricas y mecánicas, en la que se puede integrar dispositivos adicionales como sensores o motores. Este robot tiene la capacidad de desplazarse gracias a las tres unidades de accionamiento omnidireccional, catalogándolo como holónomo, es decir, que los grados de libertad controlables son iguales al número de grados de libertad totales del Robotino, el Robotino puede tener acoplado a él una torre de extensión donde puede ser ajustado un actuador, el robot tiene un diámetro de 45 cm, y una altura de 29 cm (sin la torre de extensión). Este robot posee un arreglo de sensores que están ubicados cada 40° en el perímetro del robot, en total son 9 sensores infrarrojos, gracias a estos sensores, con la respectiva programación el Robotino puede evitar obstáculos, a mayor voltaje mayor será la distancia que medirá el sensor, distancias relativas de objetos a distancias desde 3 cm hasta 40 cm.

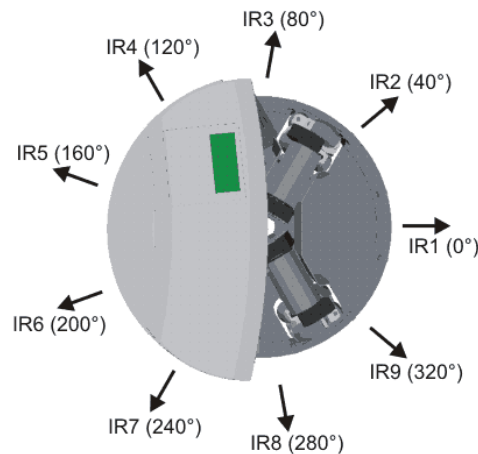


Figura 5. Ubicación sensores en el Robotino 3

Un aditamento que tiene el Robotino es la cámara Intel RealSense D435, cámara capaz de entregar imagen y video en calidad HD 1080, con láser infrarrojo para indicar la distancia de algún objeto al Robotino, cuya distancia máxima de medición es de aproximadamente 10 m, la configuración de la resolución y de otros parámetros pueden hacerse directamente desde el software del Robotino (Robotino View) o en caso tal desde otras interfaces de lenguajes de programación. Adicional a esto, el PC integrado tiene un procesador Intel Core i5, 8 GB de RAM con sistema operativo Linux Ubuntu (64 bits).

El Robotino 3 tiene un sistema de transmisión denominado Omnidrive, este se conoce más como sistema de accionamiento omnidireccional, el cual permite que el Robotino pueda trasladarse a lo largo de los dos ejes coordenados x , y . Además, permite realizar giros sobre el eje z . Con la ayuda del API del robotino, el Omnidrive permite introducir los valores que se quieren para la velocidad en el eje x , la velocidad en el eje y , la velocidad de rotación Cuenta con: motor, encoder incremental, reductor y ruedas. El sistema coordenado de referencia del robotino se localiza en su centroide, el robot rotará con él permaneciendo en su posición inicial (del movimiento traslacional) siendo la dirección x positiva donde apunte la cámara, y positiva como se muestra en la figura:

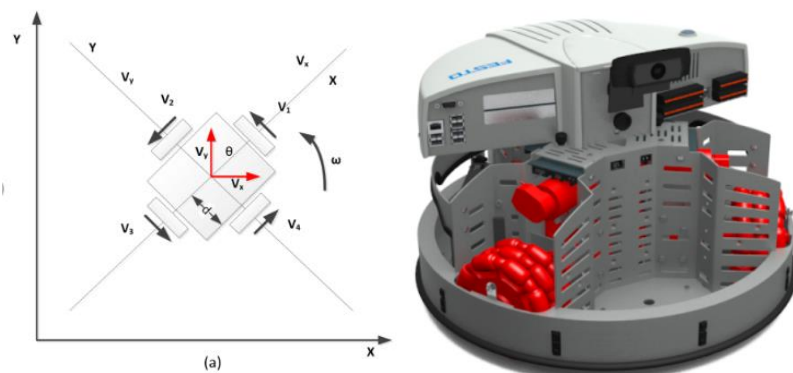


Figura 6. Sistema de referencia del Robotino y sistema Omnidrive

Los motores que utiliza el sistema omnidireccional son Dunkermotoren GR 42x40, cuya función es conducir tres ruedas omnidireccionales independientes (Ruedas que permiten mover el sistema en las direcciones y sentidos previamente mencionados), estos son conectados al output de la tarjeta de poder electrónica en el robotino, para controlarlo mediante programación se utilizan microcontroladores para dar la velocidad al motor. Cada motor está equipado con un encoder incremental, el funcionamiento de este dispositivo está basado en los valores que este motor es capaz de ajustar y regular los valores de velocidad deseada. Además, estos valores también son usados para determinar la posición del sistema móvil. La relación de transmisión es de 32:1 entre cada motor y rueda, lo que permite que el sistema móvil funcione a bajas velocidades y con alta precisión. Utilizando una correa dentada S3M también tiene un sistema de engranajes planetarios.

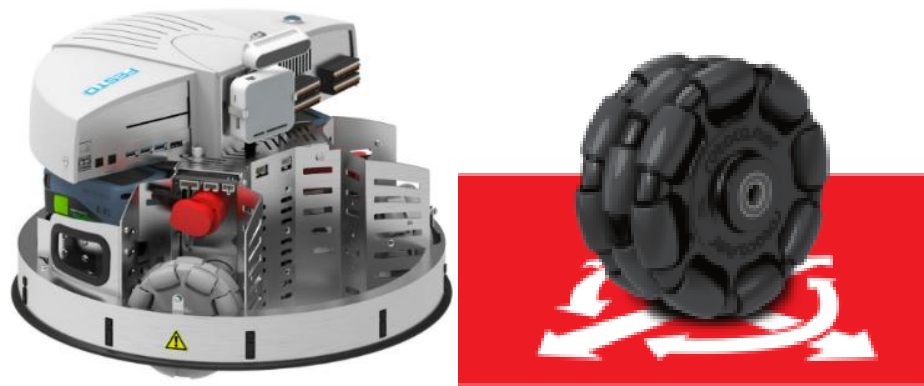


Figura 7. Ubicación Motores, encoders, reductor y representación de ruedas omnidireccionales

El software que se puede utilizar para manipular el Robotino es independiente del sistema operativo, el cual proporciona interfaces para comunicarse con él desde aplicaciones de PC externas gracias a la conexión wifi del robot. También proporciona API 2.0, librerías que facilitan la creación de aplicaciones para el Robotino en diferentes lenguajes de programación, tales como: C++, JAVA, Simulink, LabVIEW, Matlab, ROS, entre otros.

Odometría:

Es el estudio de la posición de vehículos con ruedas a través y durante su navegación. Funciona a través de la rotación de las ruedas para estimar y calcular cambios en la posición de estas a lo largo del tiempo. Esta herramienta en el robotino funciona para medir las coordenadas “x” y “y”, y la orientación en radianes del robotino en un momento determinado. Este genera un marco de referencia global al momento de ser encendido, y va a permanecer constante mientras se mantenga no sea apagado.

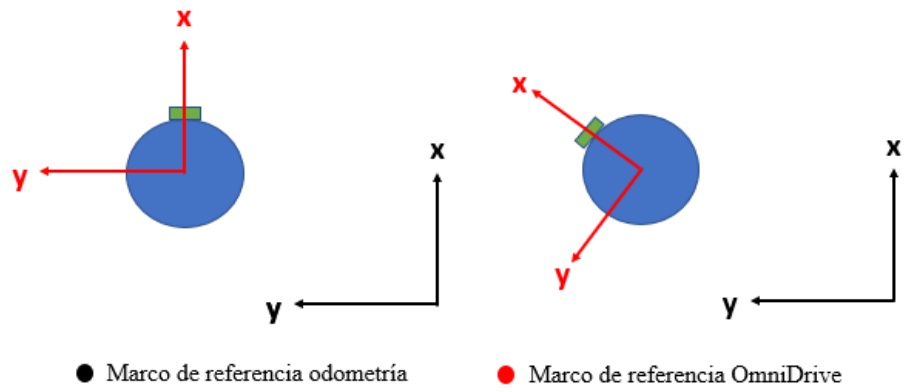


Figura 8. Diferencia entre el marco de referencia de la odometría y del OmniDrive.

Desarrollo Proyecto (Fase 1 Robocup):

En primera instancia se parametrizó y definió el ejercicio (Alcance y metodología), con la reglamentación de la RoboCup Logistics League, se realizó:

Elección lenguaje de programación

Dentro de los lenguajes de programación permitidos por el reglamento de la competición “RoboCup Logistics League” se encuentran los siguientes:

- C++ and C
- C#
- .net and JAVA
- Matlab and Simulink
- LabVIEW
- Robot Operating System (ROS)
- Microsoft Robotics Developer Studio

El proceso de elección se determinó junto al equipo de trabajo teniendo presente las siguientes pautas:

- Información en línea
- Compatibilidad con otros lenguajes
- Accesibilidad
- Facilidad de instalación del software y de sus complementos/librerías

Después de buscar información y experimentar con algunos de los programas tales como Matlab & Simulink, LabView, Python y C++

Se optó por C++, respecto a las pautas, al ser un lenguaje de programación de bajo nivel se encuentra mucho desarrollo y librerías con las cuales se pueden trabajar. Segundo, en nuestro programa de ingeniería habiendo visto introducción a la programación en lenguajes de bajo nivel sumó por la elección de este. Tercero, al contar con dispositivos no tan rápidos en términos de procesador y demás, el trabajo algo de bajo nivel facilita los procesos computacionales. Cuarto, para el implemento a futuro del programa de participación en la RoboCup a un futuro, con el asesoramiento del tutor a cargo, se determinó que un lenguaje de bajo nivel sería óptimo por las mismas razones anteriormente mencionadas, y de esta manera poder seguir con el proyecto. Por último, la guía de uso y la cantidad de librerías en este lenguaje de programación, así como el IDE, y recursos para instalación son buenos para añadir recursos al proyecto.

Descarga de librerías necesarias

Robotino API C++

Para operar el Robotino mediante el lenguaje de programación C++, se requiere instalar las librerías desarrolladas por FESTO para el Robotino, de las cuales se utilizaron la Odometría, lectura de sensores infrarrojos y láser, Bumper, “Omnidrive” movimiento del Robotino, “com.processEvents()” capturas de imagen mediante la cámara instalada en el Robotino.

OpenCV

Esta librería de uso libre o código abierto que contienen miles de algoritmos con el fin de implementar visión artificial o machine learning, en empresas, Universidades se utiliza para la detección de objetos o rostros, por ejemplo. También se especializan en aspectos de seguridad, fotografía etc. El proyecto de OpenCV fue llevado a cabo en su lenguaje principal C++. Principalmente para la aplicación en la RoboCup se utilizan las clases de Matrices y clases como “cvtColor(img, imgHSV1, COLOR_BGR2HSV)” que consiste en una transformación no lineal del espacio de colores RGB. También, el uso de recorte de Imágenes con la clase “Rect”, más el uso de Una “Máscara” que identifica un color deseado mediante píxeles blancos. Lo cual nos deja la función countNonZero() para contar el número de píxeles que está en la imagen con el color específico. Más adelante en el proceso se explicará más a fondo.

ArUco

Como una aproximación para el ingreso al RoboCup es utilizar códigos Ar que identificarán el tipo de estación. La Clase “ArucoReader” mediante vectores y recorridos de imágenes en blanco y negro puede identificar el numero (ID) perteneciente al código del Ar, así mismo, esta librería nos permite calibrar la cámara del Robotino para reconocer la posición y orientación del código Ar en el espacio respecto al Robotino.

La función “*detect*” permite imprimir y hallar todo lo contenido o anteriormente mencionado, también la librería y los proveedores de ella proporcionan libre uso de esta e imágenes con las cuales se puedan calibrar la cámara.

Uso de instrumentos del Robotino: Cámara

El robotino viene equipado con una cámara Intel RealSense D435, la cual está montada en la parte frontal del robot. Esta se utilizó para tomar fotos del espacio, y para medir distancias con el láser que trae integrado. Estas dos funciones se logran llevar a cabo gracias al API del robotino, que se ejecutan cada vez que se llama el evento `com.processEvents()`, y esto hace que se tomen y guardan imágenes en el computador tipo jpg, y al mismo tiempo, que se obtengan mediciones de distancia del láser, entre la cámara y el de algún objeto que se encuentren directamente al frente de la cámara. Esta tiene un alcance de hasta 10 metros.

El objetivo del uso de la cámara fue procesar las imágenes, y que sea posible detectar el color naranja de las cajas (MPS) y los marcadores ArUco. Y también fue esencial el uso del láser, para poder encontrar y ubicar las cajas en el espacio. Los procesos que se realizan con las imágenes tomadas por las cámaras, y el uso del láser, se explican a continuación.

Recorte de imágenes

El primer procesamiento que se le hacen a las imágenes es recortarlas de tal manera que en solo se tomara en cuenta el centro de la imagen en la identificación de color. Esto con el propósito de que se identifique el color de la caja únicamente cuando el Robotino esté posicionado de tal forma que el eje x del robotino, intersección a la caja en el momento de la detección de colores. El tamaño original de las imágenes tomadas por la cámara son de 640 x 480 píxeles, así que se decidió dividir las fotos en 5, y guardar la imagen del medio. De esta manera se van a tener imágenes de 128 x 480 píxeles. La manera como funciona en el código es especificando la región que se quiere recortar de la imagen de entrada, haciéndolo de tipo rectángulo (*Rect*). Este funciona colocando los píxeles de la esquina superior izquierda, y luego el ancho y el largo que se quiere.

Dimensiones rectángulo de la región deseada: `Rect roi (256, 0, 128, 480);`

Crear imagen de la región de la foto original: `imgCrop = img(roi);`

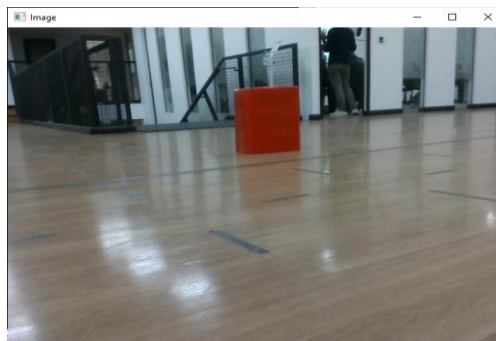


Figura 9. Tamaño de imagen tomada por la cámara del Robotino.

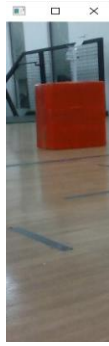


Figura 10. Imagen recortada.

Detección de colores

La detección de color juega un rol fundamental para la identificación de las cajas. Como se dijo previamente las cajas van a tener un color naranja, y este paso permite identificar cuando el robotino está orientado hacia una caja. Luego de que la imagen haya sido recortada, la imagen pasa a un post procesamiento que permite saber si el color naranja se encuentra en esa foto.

Primero se pasa la imagen de BGR a un modelo de color HSV (del inglés Hue, Saturation, Value). Esto para caracterizar los colores en términos de matiz (Hue) que representa la esencia del color, Saturación (Saturation) que representa la pureza del color y la intensidad (Value) que representa el brillo del color. Esto va a permitir escoger el color naranja, al igual que la intensidad y el brillo del naranja que se utilizó para pintar las cajas [2].

Luego se hace un umbral de la imagen en HSV. Esto lo que hace es reemplazar cada píxel de la foto a negro si la intensidad del píxel es menor a la del rango establecido dependiendo el color. O cambia el color del píxel a blanco si la intensidad del rango establecido es mayor. Para este caso se requiere identificar el color naranja de las cajas, por esto, si el rango de intensidad es específico para este color, en los pixeles donde se encontró ese tono de naranja, van a pasar a ser blancos, y el resto de los pixeles en la imagen pasaran a ser negros [3].

El rango del que depende el umbral para saber que pixeles volver blancos, y cuales negros, dependen del matiz, saturación e intensidad del color específico, y toca especificar tanto los valores mínimos como máximos para los tres parámetros. El proceso que se llevó a cabo para encontrar estos rangos para el color de las cajas fue el siguiente: Primero se tomó una foto de la caja en el espacio que se iba a utilizar, y muy importante, con la iluminación del espacio donde se llevó a cabo el trabajo. Esto

es un factor determinante porque dependiendo de la iluminación los rangos pueden cambiar. Después de esto se crea la imagen en el código, y se le realiza el cambio a HSV. Luego se crean las variables del matiz mínimo y máximo, la saturación mínima y máxima, y la intensidad mínima y máxima, que van a ser los parámetros de entrada para realizar el umbral. Para hallar los valores correctos de estas variables, se crearon controles deslizantes a cada uno, que permitan variar estos en tiempo real al correr la aplicación. De esta manera se iban moviendo los controles de estas y simultáneamente se observó el umbral de la imagen, que iba cambiando los colores de los píxeles a negro o blanco dependiendo de la posición del control deslizante. Se hizo esto hasta que se evidencio que lo único blanco en el umbral de la imagen era la región donde se encontraba la caja. Así que esos valores se guardaron, y esos son los umbrales que se utilizaron para identificar el color naranja de las cajas.

Como se había mencionado anteriormente, la iluminación juega un papel importante en el rango que se debe utilizar para la identificación de los colores, y es por esto que se realizó el mismo procedimiento, pero esta vez con una foto de la caja a contraluz. Esto fue necesario debido a que en el espacio de trabajo hay ventanas y en ciertas partes hay mayor luminosidad que en otras, y por esto el color naranja de las cajas tenía un rango diferente cuando se encontraba a contraluz. Los valores utilizados para la identificación del color normal y a contraluz se encuentran en la tabla 1.

Color	Matiz mínimo	Matiz máximo	Saturación mínima	Saturación máxima	Intensidad mínima	Intensidad máxima
Naranja (normal)	0	7	123	255	135	255
Naranja (contraluz)	0	9	113	255	40	255

Tabla 3. Rango de matiz, saturación e intensidad para los colores de las cajas.

Luego de tener el umbral imagen donde los únicos píxeles blancos corresponden a los píxeles que en la foto eran del naranja de las cajas, se hace un conteo de la cantidad de píxeles blancos que hay, esto va a permitir descartar otras cosas que puedan tener los mismos rangos HSV que las cajas, pero que no sean las cajas. La lógica que se usó fue que entre mayor cantidad de píxeles blancos mayor va a ser el tamaño del objeto naranja que se detectó. Así que, si detecto un color naranja, pero este no tiene suficientes píxeles blancos (no tiene el tamaño suficiente para ser una

caja), se descarta y no se considera una caja. El valor de píxeles blancos cuando comienza a considerarse que hay una caja en la foto fue de 750.

Conversión a HSV: `cvtColor(img, imgHSV, COLOR_BGR2HSV);`

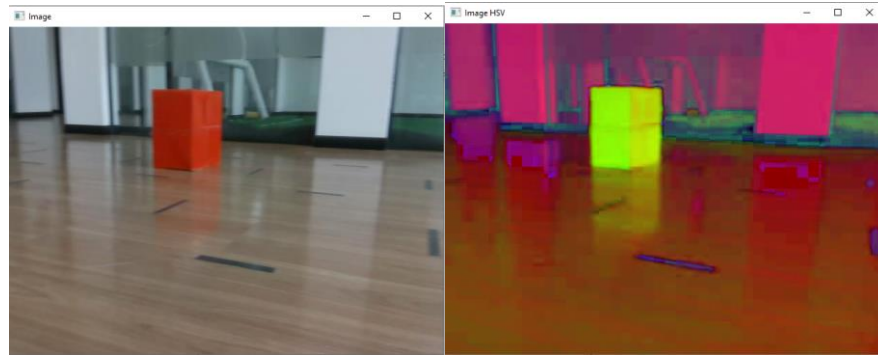


Figura 11. Conversión de la imagen a HSV

Valores escalares de los límites mínimos del HSV: `Scalar lower(hmin, smin, vmin);`

Valores escalares de los límites máximos del HSV: `Scalar upper(hmax, smax, vmax);`

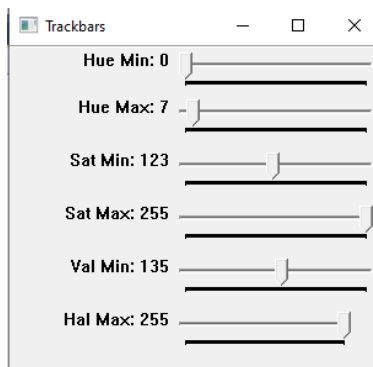


Figura 12. Valores mínimos y máximos del HSV

Umbralización de la imagen: `inRange(imgHSV, lower, upper, mask);`

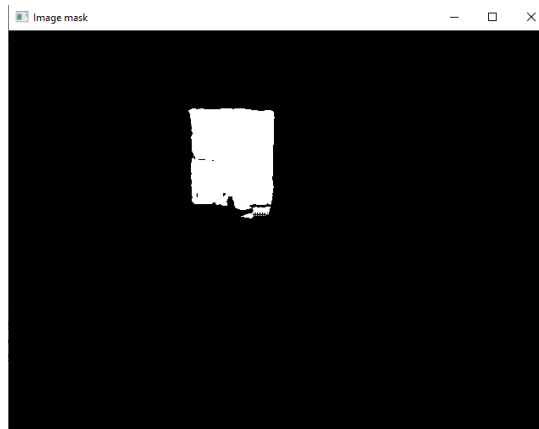


Figura 13. Umbral de la imagen HSV.

Contar píxeles blancos: $x = \text{countNonZero}(\text{mask});$

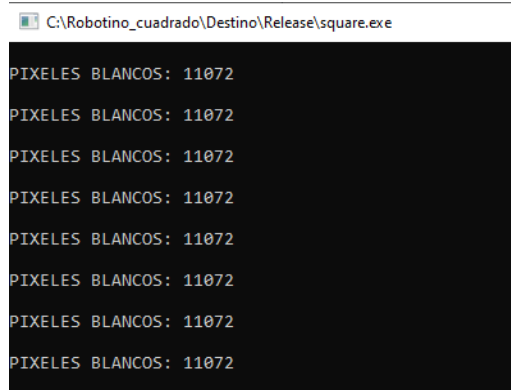


Figura 14. Cantidad de pixeles blancos en el umbral de la imagen.

Detección marcadores ArUco

La forma en que se puede diferenciar una caja de la otra es gracias a los marcadores ArUco. Cada marcador tiene su propio ID, así que se le asignó uno de estos a cada caja. Por lo tanto, la función principal del ArUco es permitir la identificación de las diferentes MPS. La manera en la que fueron identificados es gracias a la librería ArUco para C++ de tomado de [4].

Lo primero que se hizo fue la calibración de la cámara para obtener sus características. Para realizar esto, primero, se imprimió en una hoja tamaño carta, un tablero con diferentes códigos ArUco (llamado aruco_calibration_grid_board_a4.pdf) [5].

Esta hoja se pegó a un pedazo de cartón ya que para que la calibración sea correcta el tablero debe ser totalmente plano. Se implementó un código en C++ para tomar fotos con la cámara del robotino. Se tomaron fotos a diferentes distancias y diferentes inclinaciones, moviendo y rotando el tablero en los 3 ejes. Se tomaron 52 fotos asegurando que el tablero completo apareciera en la imagen. Todas las fotos se guardaron en una carpeta, y ya con esto se utilizó un programa llamado `aruco_calibration_fromimages.cpp` [4] para realizar la calibración.

Para ejecutar este fue necesario introducir la dirección de la carpeta con las imágenes del tablero, un archivo YML donde se guardaron los resultados y el tamaño de un solo marcador ArUco del tablero en metros, en este caso tenían un tamaño de 0.0385 metros.

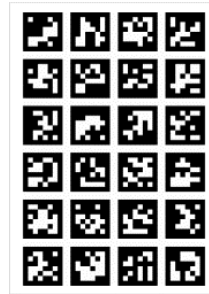


Figura 15. Tablero de ArUcos para la calibración de la cámara.

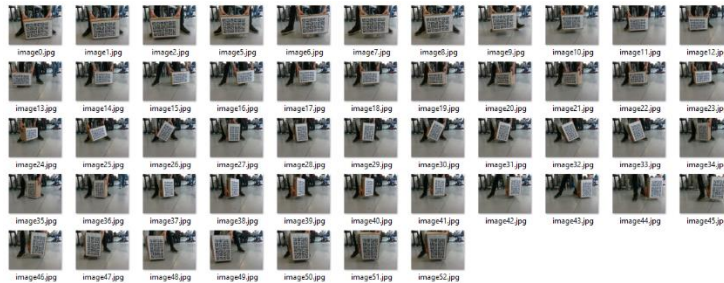


Figura 16. Fotos tomadas para la calibración de la cámara.

Una vez hecha la calibración de la cámara, y ya teniendo el archivo con sus características, se puede realizar la detección de los marcadores. Esto se realiza iniciando un objeto de la clase *MarkerDetector*, que tiene una función llamada *detect*, la cual permite guardar un vector con todos los marcadores ArUco que se encuentren. Otra utilidad de esta función es que permite encontrar la orientación del marcador. Los ArUco tienen su propio marco de referencia, y con la librería se puede hallar los vectores de rotación y traslación, del marco de referencia de la cámara con respecto a la del marcador identificado. Al utilizar la función *detect* se pide como parámetros de entrada la imagen donde se van a buscar los marcadores, un archivo YML con los datos de la calibración de la cámara, y el tamaño de los Arucos que se

van a detectar en metros, en este caso los marcadores fueron de 0.1 metros. La función utiliza esta información para hallar los valores de traslación y de rotación en los ejes x, y, z, y los guarda en dos vectores. En este caso solo fue necesario utilizar la orientación en el eje y del marcador (Los ejes se evidencian en la figura 17), que permitió saber que orientación tenía el marcador en el marco de referencia del proyecto.



Figura 17. Marco referencia del marcador ArUco

```
[ -2.6522133;  
  -0.084090732;  
   1.2987511] [0.0078920396;  
   0.011045927;  
   0.33281589]
```

Figura 18. Vector de rotación y de traslación del marco de referencia del ArUco con respecto al marco de referencia de la cámara

Laser rangefinder

La cámara Intel RealSense D435 cuenta con un proyector infrarrojo que sirve para medir distancias desde 0.3 a 10 metros (dependiendo de la iluminación) [6].

Parameter	Properties
Projector	Infrared
Pattern Type	Static
Illuminating Component	Vertical-cavity surface-emitting laser (VCSEL) + Optics
Laser Controller	PWM
Optical Power	360mW average, 440mW peak
Laser Wavelength	850nm $\pm$ 10 nm nominal @ 20°C
Laser Compliance	Class 1, IEC 60825-1:2007 Edition 2, IEC 60825-1:2014 Edition 3
Horizontal Field of Projection	64° $\pm$ 3°
Vertical Field of Projection	41° $\pm$ 3°
Diagonal Field of Projection	72° $\pm$ 3°

Tabla 4. Características del proyecto laser de la cámara

Las mediciones que realiza este láser son gracias a la clase LaserRangeFinder del API del robotino. En este se define que cada vez que se llame el evento `com.processEvents()`, se realice una medición de distancia con el láser. Este número va a estar en metros, y si no detecta nada en menos de 10 metros, imprime un 0.

Sensores Infrarrojos

El Robotino cuenta con 9 sensores infrarrojos alrededor del robotino en la parte inferior, distribuidos cada 40°. Sirven para identificar si hay algo próximo al robotino, si se encuentra entre 4 y 30 cm. La manera en que se utilizaron estos sensores fue creando un arreglo de 9 `DistanceSensor`, que es una clase del API. En este se guardan los voltajes de los nueve sensores, siendo la posición 0 del arreglo el sensor frontal del robotino, y siguen los sensores en sentido antihorario. Así que las posiciones en el arreglo de cada uno de los sensores son como se muestra en la siguiente figura:

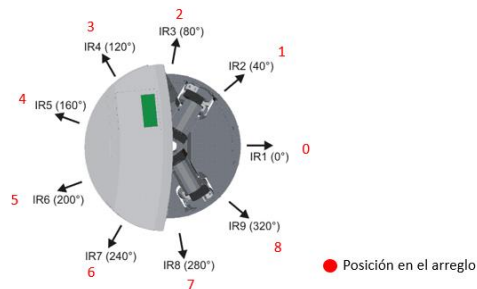
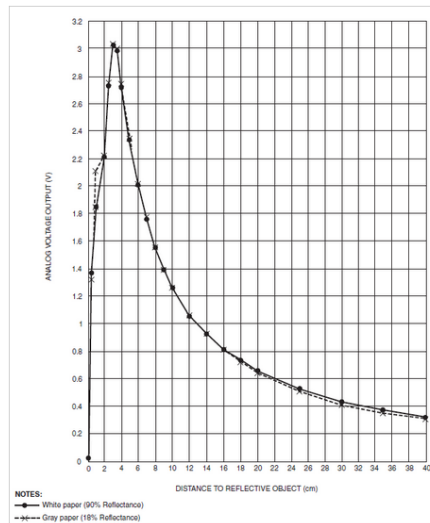


Figura 19. Número y posición de los sensores del Robotino en el arreglo del código.

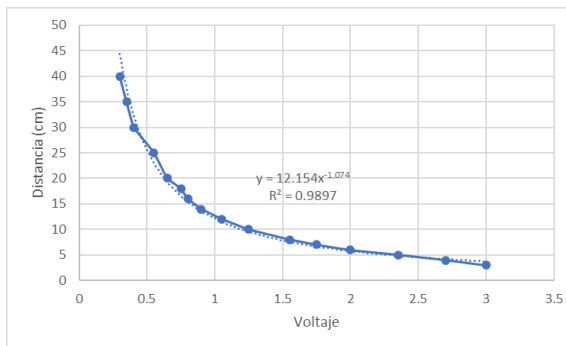
Los valores que se reciben en el computador son de voltaje, así que para poder conocer las distancias que equivalen a esos voltajes, se utilizó la gráfica 1 (obtenida del *datasheet* [8]) de los sensores, para encontrar la función que más se ajustara a la relación entre el voltaje y la distancia en centímetros. La función que más se ajustó fue la relación potencial:

$$D = 12.154 * v^{-1.074}$$

Donde D hace referencia a la distancia hasta un objeto y v al voltaje análogo de salida.



Gráfica 1. Distancia [cm] vs Voltaje [V]



Gráfica 2. Regresión no lineal.

Disposición del área de trabajo

Primero se definió el área en el que se llevaría a cabo el proyecto, que resultó ser un espacio de 7 m x 2.8 m. Para realizar la ubicación de las cajas, se decidió dividir el espacio en cuadrantes de 70 centímetros, para así crear una cuadrícula de 10 filas por 4 columnas (para un total de 40

cuadrantes), que facilita guardar las posiciones de las cajas, y la subsecuente búsqueda de estas. El robotino empezará en el cuadrante inferior derecho, y el marco de referencia que se va a tomar, es con x positivo en dirección de las columnas, y positivo en dirección de las filas. Para estar acorde al marco de referencia generada por la odometría del API del robotino, la cámara del robotino se coloca mirando en dirección a las columnas. Así el origen del marco de referencia va a ser en el centro del cuadrante en el que inicia el robotino.



Figura 20. Zona de trabajo delimitada

La manera en que la cuadrícula va a funcionar es marcando los cuadrantes que están vacíos con un 1, y marcando los cuadrantes que contienen una caja con un 3. De esta manera, se sabrán las posiciones a las que debe ir, y los lugares donde puede avanzar sin colisionar con alguna estación.

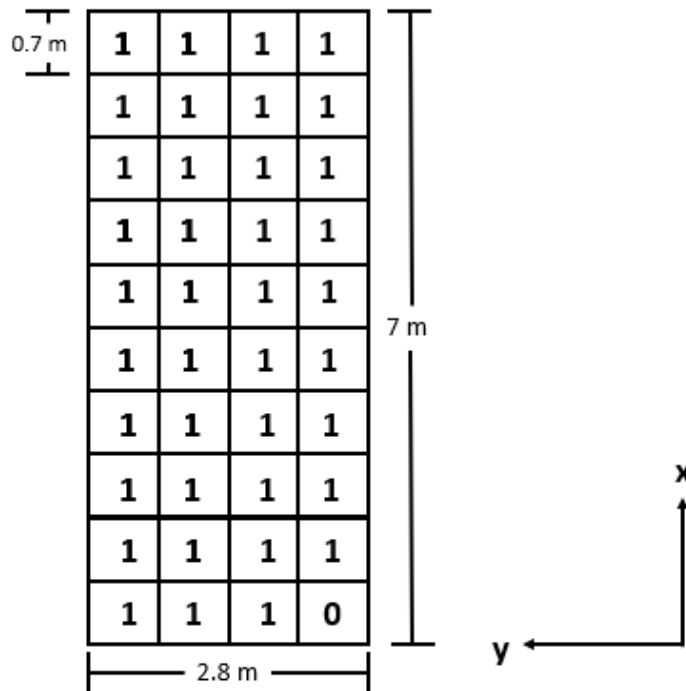


Figura 21. Cuadrícula con cuadrantes

Para realizar esta representación digitalmente, primero, se creó una matriz de 10x4 llena de unos, queriendo decir que todos los cuadrantes están vacíos. Esta matriz va a representar la cuadrícula, donde la posición que contenga un 1, es un cuadrante vacío, y uno que tenga un 3 es que contiene una caja. A esta matriz se le denomino *grid*.

```
array<array<int, 4>, 10> grid{
    { { 1, 1, 1, 1 } },
    { { 1, 1, 1, 1 } },
    { { 1, 1, 1, 1 } },
    { { 1, 1, 1, 1 } },
    { { 1, 1, 1, 1 } },
    { { 1, 1, 1, 1 } },
    { { 1, 1, 1, 1 } },
    { { 1, 1, 1, 1 } },
    { { 1, 1, 1, 1 } },
    { { 1, 1, 1, 1 } },
    { { 1, 1, 1, 1 } }, }
};
```

Figura 22. Matriz representando la cuadrícula

También se creó una clase llamada *cuadrant*, que representa un cuadrante de la cuadrícula. Esta clase tiene como parámetros la coordenada del límite inferior en x, el límite inferior en y, el límite superior en x, el límite superior en y, la columna, y la fila de la matriz. Luego se crea un

arreglo de 40 elementos tipo *cuadrant*, que son los cuadrantes totales en nuestro espacio. Y se emplea una función para especificar los parámetros de cada cuadrante en el arreglo. Las columnas y las filas empiezan en el cuadrante superior izquierdo, siendo el cuadrante 0, 0. Y termina en el inferior derecho, siendo este el 9, 3.

Algoritmo de búsqueda

Ahora, el método de búsqueda implementado para buscar las coordenadas y orientación de los marcadores ArUco, se puede dividir en 3 momentos. Primero, el robotino va a encontrar, desde su posición inicial, en qué cuadrantes están ubicadas las posibles cajas. Segundo, se va a ir al cuadrante más cercano donde se haya estimado que hay una caja, en este paso se verifica si en realidad hay una caja en ese cuadrante. Y, por último, se recorre la caja para buscar y leer el marcador ArUco. Una vez leído el marcador, el Robotino seguirá al siguiente cuadrante hasta que inspeccione todos los cuadrantes donde se marcaron que había cajas en el primer paso. Si al principio no se encontraron las 3 cajas, se realizará una operación para buscar las cajas que faltan. Estos pasos son explicados en profundidad a continuación.

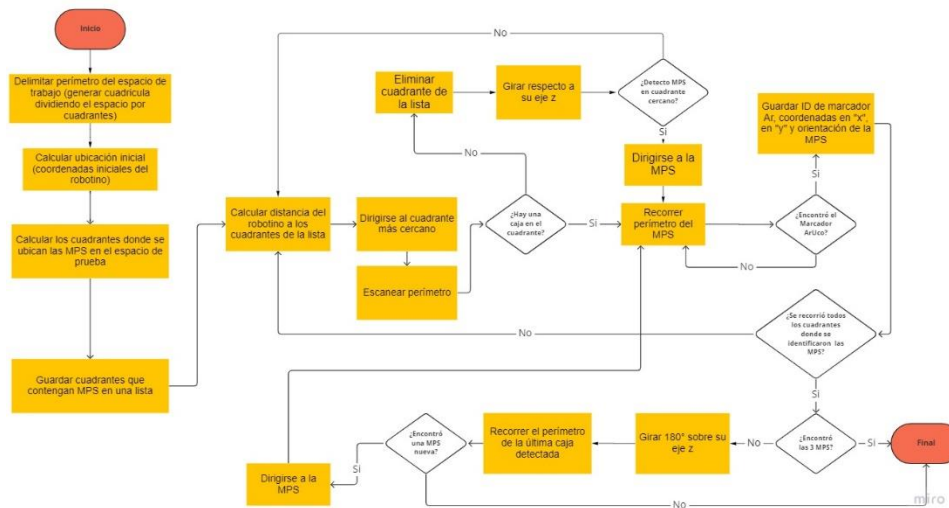


Diagrama 1. Diagrama de flujo del algoritmo de búsqueda.

Momento 1. Encontrar cuadrante de las cajas

En este paso se va a realizar una estimación de las posiciones de las cajas en el espacio, sin necesidad de que el robotino se traslade de su posición inicial. Esto va a funcionar haciéndolo rotar sobre su propio eje de tal manera que el lente de la cámara pueda estar de frente hacia todo el espacio. La manera en que sirve es gracias al láser

de la cámara del robotino, que halla la distancia que hay entre el robotino y las cajas, y también se usa la orientación del robotino que da el odómetro. Con estos dos se puede estimar en qué cuadrante se encuentran las cajas utilizando identidades trigonométricas.

Se evidenció que el láser puede medir distancias erróneas debido a ruido en la señal, por lo que, si solo se toma una medición, puede que se ubique la caja en el cuadrante equivocado. Para solucionar esto se decidió que, en lugar de solo utilizar una medida, se van a encontrar las coordenadas de la caja, utilizando el promedio de distancias medidas por el láser desde que encuentra la caja hasta que la pierde. Esto también va para el ángulo del robotino, donde se utiliza el promedio del ángulo en el momento de encontrar la caja, y del ángulo el momento que la pierde. Por esto las ecuaciones que se usaron para estimar las coordenadas de la caja fueron las siguientes:

$$x_{caja} = pd * \cos(pa)$$

$$y_{caja} = pd * \sin(pa)$$

Donde pd hace referencia al promedio entre distancias y pa hace referencia al promedio entre ángulos. A parte de hallar el promedio de las distancias, también se tomó otra medida para que la estimación del cuadrante fuera lo más correcta posible, que fue descartar los valores atípicos dentro de las distancias medidas para una caja, utilizando el método del rango intercuartílico. Los valores atípicos son las mediciones que se alejan de manera considerable al resto de datos, y en este caso, esto puede causar que el cuadrante donde se estima que hay una caja, esté en realidad equivocado. Para evitar esto se implementó el método del rango intercuartílico, que consistió en, primero, ordenar todas las medidas del láser de menor a mayor. Segundo, se halla el cuartil 1 (Q1) y el cuartil 3 (Q3), que es la mediana de su respectivo cuartil. Ya con estos valores se puede hallar el rango Inter cuartil (IQR), que es la resta entre Q3 Y Q1. Luego se calcula el promedio de los datos y por último se toma el valor más grande y se le resta el promedio, si este es mayor a 1.5 veces el IQR, se dice que es un valor atípico y se debe eliminar de los datos. Igual pasa con el valor más pequeño, si al restar este al promedio da menor que 1.5 veces el IQR, también se considera un valor atípico y no se debe tener en cuenta.

$$IQR = Q3 - Q1$$

Valores atípicos

$$Dato\ más\ grande - Promedio > 1.5 * IQR$$

$$\text{Promedio} - \text{Dato más pequeño} < 1.5 * IQR$$

Una vez excluidos los valores atípicos (si es el caso), se halla el promedio de las distancias medidas por el láser, y con ese valor, y el promedio de los ángulos si se estima las coordenadas de la caja.

Lo anterior comprende la primera parte de la búsqueda de los códigos ArUco, posterior a esto lo que se hace es medir las coordenadas y orientación del robotino utilizando la odometría. Como las medidas por el odómetro son internas del computador del robotino, estas no van a ser 0 cada vez que se corra el programa. Por esto las primeras medidas se restarán a todas las mediciones futuras para asegurar que el robotino comience en las coordenadas (0, 0), y que estas queden en el centro del cuadrante 9, 3 donde se definió el origen de nuestro sistema de referencia. Esto permitirá que todas las mediciones futuras sean correctas y de acuerdo con el mismo sistema de referencia y origen.

Luego, se hará que el robotino gire 90 grados sobre su propio eje, a una velocidad de 0.1 rad/s utilizando la función de setVelocity de la clase Omnidrive del API del robotino. Esto se hace para que la cámara, y el láser que contiene, pueda “observar” todo el espacio, y que mientras rote, se identifiquen las cajas por su color, y que el láser realice mediciones de distancia para estimar en qué cuadrante se encuentran.

Al mismo tiempo que el robotino rota, se van a tomar fotos, que se guardan, y recortan para realizar la identificación del color naranja. Y también se van a realizar mediciones de distancia con el láser. Ambas ocurren cada 100 milisegundos. Mientras gira, se va a decir que se identificó una caja si el láser realiza una medición mayor a 0 o menor a 6.5 (que es la distancia máxima a la que puede estar una caja del robotino) y si se identifica el color naranja de la caja. Una vez se cumplan estas condiciones, las distancias que se midan van a ser guardadas en una lista hasta que se considere que se perdió la caja (cuando el láser deje de reflejarse en la caja), en ese momento se acaba la lista. Los parámetros que se consideraron para determinar que ya dejó de tomar medidas de la caja, es si la distancia medida por el láser es igual a 0, si hay 1 metro de diferencia entre la última medición y la distancia anterior, o si se deja de detectar el color naranja. Es importante aclarar que no todas las medidas que se toman durante el periodo en el que se detecta una caja son incluidas en la lista, para que estas sean contadas, deben ser mayores a 0, y que las distancias estén dentro del espacio de trabajo dependiendo en qué grado de orientación se encuentre el robotino. Para descartar estos valores se determinaron funciones para saber cuál va a ser la distancia máxima permitida ($Dist_{max}$) en relación con el ángulo actual del robotino (α). Para realizar esto se tuvieron que hacer 2, una para cuando el robotino solo haya rotado 20.2° y otra para el resto del camino.

Distancia máxima permitida cuando el ángulo actual es menor a 20.2°

$$\alpha > 20.2^\circ \quad Dist_{max} = \frac{2.45}{\cos(\pi/2 - \alpha)}$$

Distancia máxima permitida cuando el ángulo actual es mayor a 20.2°

$$\alpha < 20.2^\circ \quad Dist_{max} = \frac{6.65}{\cos(\alpha)}$$

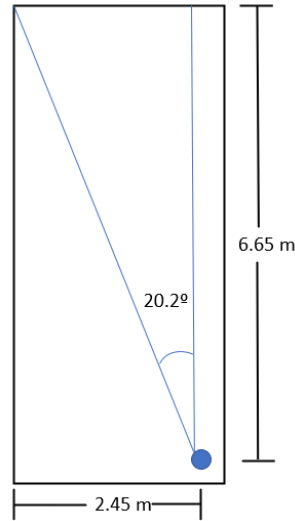


Figura 23. Representación Distancias máximas permitidas

Ya al tener la lista completa de las distancias que se obtuvieron para una caja, se le eliminan los valores atípicos, con el método del rango intercuartílico explicado anteriormente, se saca el promedio de la lista, y el promedio de los ángulos de cuando se empezó a detectar la caja y de cuando se terminó. Con estos dos valores se hallan las coordenadas x y y de la caja, y una vez teniendo este, se recorre el arreglo de cuarenta cuadrantes para verificar en qué cuadrante se encuentran esas coordenadas. Una vez determinado el cuadrante en el que se halla, se va a marcar con un 3 en la matriz de la cuadrícula (*grid*). Este proceso se repite para todas las cajas que encuentre el robotino mientras gira los 90 grados. En la Figura 24 se ve un ejemplo de cómo puede quedar el *grid* después del proceso.

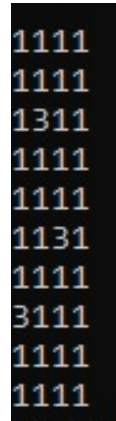


Figura 24. Escenario con estaciones registradas en columna 0 fila 7, columna 1 fila 2 y columna 2 fila 6

Una vez terminada la rotación, se va a recorrer la matriz *grid*, y se van a guardar en una lista los cuadrantes que estén marcados con un tres.

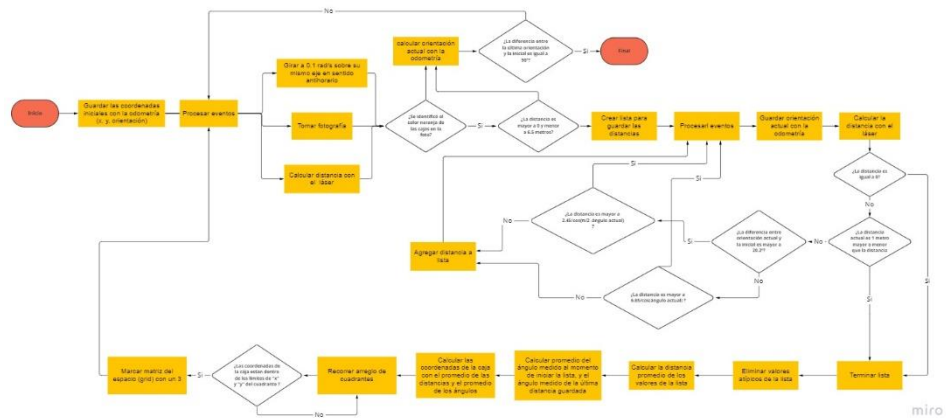


Diagrama 2. Diagrama de flujo momento 1.

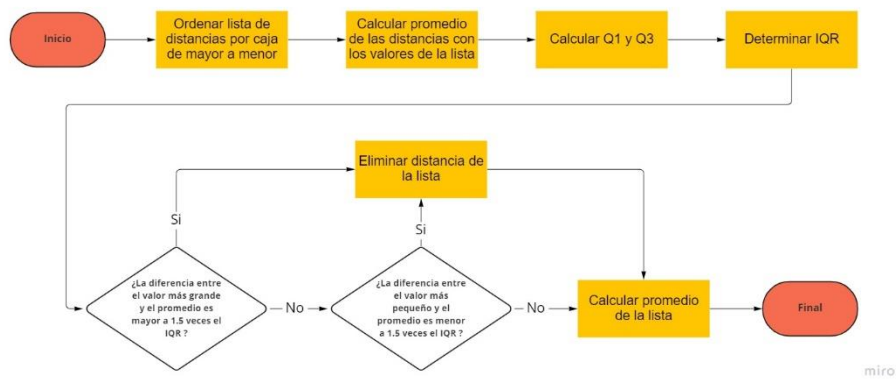


Diagrama 3. Diagrama de flujo eliminación de valores atípicos.

Momento 2. Ir al cuadrante con caja más cercano

Esta etapa tiene el objetivo de encontrar el cuadrante más cercano al robotino que contenga una caja, para ir a dicho cuadrante para verificar que de verdad se encuentre ahí una caja, y luego buscar el ArUco.

De esta manera, lo primero que se hace es recorrer la lista de los cuadrantes con cajas, y comparar las distancias entre cada uno de esos cuadrantes y la posición actual del robotino. Cabe aclarar que como esto se va a realizar para cada caja identificada en el primer paso, solo se van a tener en cuenta las cajas que no han sido revisadas. Para hacer esto, en el momento en que se identifica el marcador en una caja, este se dice que es procesado, y ya no se tiene en cuenta para este proceso. Una vez encontrado el cuadrante más cercano, se dice que ese es el objetivo al que tiene que ir el robotino, y que se debe mover para allá.

Para hacer que el robotino vaya al destino deseado se utiliza la función de *setVelocity* del Omnidrive, donde, primero se hace que rote a su orientación original a 0.2 rad/s, y luego se colocan las velocidades en x y en y necesarias para llegar al cuadrante objetivo en el menor tiempo posible. Este proceso comienza por hallar las distancias en x y en y que hay entre la posición actual del robotino y las coordenadas centrales del cuadrante al que tiene que ir, esto va a permitir que el robotino vaya en la dirección correcta y de manera directa a su destino. Estas distancias son las que van a dictar qué velocidades se colocan en el Omnidrive. Lo que se hizo fue determinar unos factores que dividan aquellas distancias con el objetivo de introducir esos cocientes como las respectivas velocidades en x y en y . Algo importante en este punto es que se identificó con varias iteraciones que la velocidad óptima no puede sobrepasar los 0.4 m/s. Esto para que el robotino vaya lo más rápido posible, pero que la velocidad no sea tan alta para que las ruedas omnidireccionales se empezaran a deslizar, causando cambios en la dirección. Para asegurarse de que los valores por los cuales se dividían las distancias en x y y , resultaran en valores adecuados de velocidad, se determinaron que los factores dependieran de que tan grandes fueran las distancias, de este modo se eligieron 3 factores para los siguientes casos:

Rango de distancia (metros)	Factor divisor de la distancia
$x \geq 5$	18
$3 \leq x < 5$	12
$x < 3$	8

Tabla 5. Factores reducción/aumento velocidad

Cabe aclarar que tanto la distancia x como la y se dividen por el mismo factor, para que la dirección que toma el robotino sea el adecuado, y que este no se vea afectado.

Una vez sabiendo las distancias, también se sabe en qué dirección tiene que ir, si es al noroccidente, al suroccidente, suroriente o nororiente. Esto es importante saberlo en caso de que se encuentre una caja ya procesada en su camino y sea necesario realizar un movimiento para esquivarla. Para saber si esto es necesario, se utilizarán los sensores infrarrojos para determinar si hay una caja en el camino. Una vez determinada la dirección que debe tomar, se revisarán los sensores que estén orientados hacia la dirección que el robotino tiene que ir. Si estos miden un voltaje mayor a 1 voltio, que significa que la caja está más cerca de 13 centímetros, se hace que el robotino se mueva 70 centímetros de forma horizontal o vertical, de tal manera que el camino quede libre. Dependiendo de la dirección que se debe tomar, los sensores que se revisan para cada posibilidad se muestran en la tabla 6.

Dirección	Sensores que revisar
Noroccidente	0, 1, 2, 3, 8
Suroccidente	2, 3, 4, 5
Suroriente	4, 5, 6, 7
Nororiente	0, 1, 6, 7, 8

Tabla 6. Dirección y sensores

De ser necesario que el robotino haga los movimientos para esquivar la caja, se calcula la nueva posición en la que está, la nueva distancia que tiene que recorrer, y se calculan nuevas velocidades en x y y .

Después de esto el robotino ya está listo para ir al cuadrante objetivo en busca de la caja. Lo primero es colocar las velocidades halladas en el Omnidrive, y este va a parar una vez llegue al cuadrante meta, o si llega a encontrar la caja con los sensores infrarrojos. Como quería que el robotino haga todo el proceso lo más rápido posible, se pone el robotino a andar a la velocidad máxima posible, como ya se había mencionado anteriormente, sin embargo, como va con tanta velocidad, el robot puede llegar a chocar con las cajas y detenerse ya cuando es muy tarde. Así que se hizo que la velocidad disminuyera a la mitad justo cuando se encontrara a menos de 70 centímetros. En este espacio, es cuando también empiezan a funcionar los

Si se encuentra la caja con los sensores, significa que la estimación del primer paso fue correcta, y que se debe rodear la caja para encontrar el marcador. Sin embargo, puede ocurrir que la estimación haya sido errónea, y que el robotino llegue al cuadrante y no encuentre nada. Si llega a suceder esto se escribió una función para revisar si la caja se encuentra en realidad en uno de los cuadrantes vecinos del que está. La forma en que hace esta revisión es girando sobre su eje a 0.15 rad/s , y durante ese giro va a realizar mediciones con el láser y a tomar fotos, y si alguna distancia medida es mayor a 0, menor a 1 metro, y se identificó el color naranja, se va a considerar que se encontró la caja, así que va a parar de girar, y va a empezar a andar en dirección de la caja encontrada hasta que los sensores infrarrojos detecten la caja, que va a ser el mismo voltaje que la vez pasada. Cuando esto ocurre, el 3 del cuadrante donde llegó y no había caja es borrado, y marca con un 3 el cuadrante en el que en realidad está la caja.

```

graph TD
    Inicio([Inicio]) --> B1[Calcular coordenadas del cuadrante al que debe ir el Robo (cuadrante objetivo)]
    B1 --> B2[Orar el roboto a su orientación inicial]
    B2 --> B3[Guardar coordenadas actuales con la codonría (x, y, orientación)]
    B3 --> B4[Calcular distancias en "x" y "y" entre las coordenadas del Robo y el cuadrante objetivo]
    B4 --> B5[Calcular velocidad en "x" y "y"]
    B5 --> B6[Realizar lectura de los voltajes de los sensores infrarrojos]
    B6 --> D1{¿los sensores detectan obstáculo a 1 metro?}
    D1 -- No --> B7[Desplazar el roboto a 0.2 m/s]
    D1 -- Si --> B8[Desplazar el roboto a la velocidad calculada]
    B7 --> B9{¿la distancia entre las coordenadas actuales y el cuadrante objetivo es menor a 0.1 metros?}
    B8 --> B9
    B9 -- No --> B10[Calcular distancia en "x" y "y" entre las coordenadas del Robo y el cuadrante objetivo]
    B9 -- Si --> B11[Avanzar con la velocidad en "x" y "y" a la mitad]
    B10 --> B12[Guardar coordenadas actuales con la codonría (x, y, orientación)]
    B11 --> B12
    B12 --> B13[Guardar ID y vector de rotación del mano de referencia del ARJOS respecto a la referencia de la cámara]
    B13 --> B14[Calcular coordenadas de la MPS]
    B14 --> B15[Calcular orientación del ARJOS respecto al mano de referencia global]
    B15 --> B16[Imprimir ID, cuadrante de la MPS (objetivo), coordenadas de su mano, coordenadas de la MPS y su orientación]
    B16 --> B17[¿la distancia entre las coordenadas actuales y el cuadrante objetivo es menor a 1 metro?]
    B17 -- No --> B18[Guardar coordenadas actuales con la codonría (x, y, orientación)]
    B17 -- Si --> B19{¿el robot se ha movido un giro completo?}
    B18 --> B19
    B19 -- Si --> B20[Final]
    B19 -- No --> B21[¿la distancia entre las coordenadas actuales y el cuadrante objetivo es menor a 0.1 metros?]
    B21 -- Si --> B22[Orar respecto a su ID 2]
    B21 -- No --> B23[Procesar eventos]
    B22 --> B23
    B23 --> B24[Tomar fotografía]
    B24 --> B25{¿se identificó el giro correcto de las cámaras?}
    B25 -- Si --> B26[Calcular distancia con el laser]
    B25 -- No --> B27[Calcular distancia con el laser]
    B26 --> B28[Orar respecto a su ID 2]
    B27 --> B28
    B28 --> B29[Marcar con "x" el cuadrante objetivo en la cuadrícula]
    B29 --> B30{¿las coordenadas actuales son iguales a las coordenadas del cuadrante objetivo?}
    B30 -- Si --> B31[Guardar coordenadas actuales con la codonría (x, y, orientación)]
    B30 -- No --> B32{¿los sensores infrarrojos detectan un obstáculo a 1 metro?}
    B31 --> B32
    B32 -- Si --> B33[Recorner perímetro del MPS]
    B32 -- No --> B34{¿la distancia entre las coordenadas actuales y el cuadrante objetivo es menor a 0.1 metros?}
    B34 -- Si --> B35[Calcular distancia con el laser]
    B34 -- No --> B36[Orar respecto a su ID 2]
    B35 --> B36
    B36 --> B37[Procesar eventos]
    B37 --> B38[Tomar fotografía]
    B38 --> B39{¿se identificó el giro correcto de las cámaras?}
    B39 -- Si --> B40[Calcular distancia con el laser]
    B39 -- No --> B41[Calcular distancia con el laser]
    B40 --> B42[Orar respecto a su ID 2]
    B41 --> B42
    B42 --> B43[Marcar con "x" el cuadrante objetivo en la cuadrícula]
    B43 --> B44{¿las coordenadas actuales son iguales a las coordenadas del cuadrante objetivo?}
    B44 -- Si --> B45[Guardar coordenadas actuales con la codonría (x, y, orientación)]
    B44 -- No --> B46{¿los sensores infrarrojos detectan un obstáculo a 1 metro?}
    B45 --> B46
    B46 -- Si --> B47[Recorner perímetro del MPS]
    B46 -- No --> B48{¿la distancia entre las coordenadas actuales y el cuadrante objetivo es menor a 0.1 metros?}
    B48 -- Si --> B49[Calcular distancia con el laser]
    B48 -- No --> B50[Orar respecto a su ID 2]
    B49 --> B50
    B50 --> B51[Procesar eventos]
    B51 --> B52[Tomar fotografía]
    B52 --> B53{¿se identificó el giro correcto de las cámaras?}
    B53 -- Si --> B54[Calcular distancia con el laser]
    B53 -- No --> B55[Calcular distancia con el laser]
    B54 --> B56[Orar respecto a su ID 2]
    B55 --> B56
    B56 --> B57[Marcar con "x" el cuadrante objetivo en la cuadrícula]
    B57 --> B58{¿las coordenadas actuales son iguales a las coordenadas del cuadrante objetivo?}
    B58 -- Si --> B59[Guardar coordenadas actuales con la codonría (x, y, orientación)]
    B58 -- No --> B60{¿los sensores infrarrojos detectan un obstáculo a 1 metro?}
    B59 --> B60
    B60 -- Si --> B61[Recorner perímetro del MPS]
    B60 -- No --> B62{¿la distancia entre las coordenadas actuales y el cuadrante objetivo es menor a 0.1 metros?}
    B62 -- Si --> B63[Calcular distancia con el laser]
    B62 -- No --> B64[Orar respecto a su ID 2]
    B63 --> B64
    B64 --> B65[Procesar eventos]
    B65 --> B66[Tomar fotografía]
    B66 --> B67{¿se identificó el giro correcto de las cámaras?}
    B67 -- Si --> B68[Calcular distancia con el laser]
    B67 -- No --> B69[Calcular distancia con el laser]
    B68 --> B70[Orar respecto a su ID 2]
    B69 --> B70
    B70 --> B71[Marcar con "x" el cuadrante objetivo en la cuadrícula]
    B71 --> B72{¿las coordenadas actuales son iguales a las coordenadas del cuadrante objetivo?}
    B72 -- Si --> B73[Guardar coordenadas actuales con la codonría (x, y, orientación)]
    B72 -- No --> B74{¿los sensores infrarrojos detectan un obstáculo a 1 metro?}
    B73 --> B74
    B74 -- Si --> B75[Recorner perímetro del MPS]
    B74 -- No --> B76{¿la distancia entre las coordenadas actuales y el cuadrante objetivo es menor a 0.1 metros?}
    B76 -- Si --> B77[Calcular distancia con el laser]
    B76 -- No --> B78[Orar respecto a su ID 2]
    B77 --> B78
    B78 --> B79[Procesar eventos]
    B79 --> B80[Tomar fotografía]
    B80 --> B81{¿se identificó el giro correcto de las cámaras?}
    B81 -- Si --> B82[Calcular distancia con el laser]
    B81 -- No --> B83[Calcular distancia con el laser]
    B82 --> B84[Orar respecto a su ID 2]
    B83 --> B84
    B84 --> B85[Marcar con "x" el cuadrante objetivo en la cuadrícula]
    B85 --> B86{¿las coordenadas actuales son iguales a las coordenadas del cuadrante objetivo?}
    B86 -- Si --> B87[Guardar coordenadas actuales con la codonría (x, y, orientación)]
    B86 -- No --> B88{¿los sensores infrarrojos detectan un obstáculo a 1 metro?}
    B87 --> B88
    B88 -- Si --> B89[Recorner perímetro del MPS]
    B88 -- No --> B90{¿la distancia entre las coordenadas actuales y el cuadrante objetivo es menor a 0.1 metros?}
    B90 -- Si --> B91[Calcular distancia con el laser]
    B90 -- No --> B92[Orar respecto a su ID 2]
    B91 --> B92
    B92 --> B93[Procesar eventos]
    B93 --> B94[Tomar fotografía]
    B94 --> B95{¿se identificó el giro correcto de las cámaras?}
    B95 -- Si --> B96[Calcular distancia con el laser]
    B95 -- No --> B97[Calcular distancia con el laser]
    B96 --> B98[Orar respecto a su ID 2]
    B97 --> B98
    B98 --> B99[Marcar con "x" el cuadrante objetivo en la cuadrícula]
    B99 --> B100{¿las coordenadas actuales son iguales a las coordenadas del cuadrante objetivo?}
    B100 -- Si --> B101[Guardar coordenadas actuales con la codonría (x, y, orientación)]
    B100 -- No --> B102{¿los sensores infrarrojos detectan un obstáculo a 1 metro?}
    B101 --> B102
    B102 -- Si --> B103[Recorner perímetro del MPS]
    B102 -- No --> B104{¿la distancia entre las coordenadas actuales y el cuadrante objetivo es menor a 0.1 metros?}
    B104 -- Si --> B105[Calcular distancia con el laser]
    B104 -- No --> B106[Orar respecto a su ID 2]
    B105 --> B106
    B106 --> B107[Procesar eventos]
    B107 --> B108[Tomar fotografía]
    B108 --> B109{¿se identificó el giro correcto de las cámaras?}
    B109 -- Si --> B110[Calcular distancia con el laser]
    B109 -- No --> B111[Calcular distancia con el laser]
    B110 --> B112[Orar respecto a su ID 2]
    B111 --> B112
    B112 --> B113[Marcar con "x" el cuadrante objetivo en la cuadrícula]
    B113 --> B114{¿las coordenadas actuales son iguales a las coordenadas del cuadrante objetivo?}
    B114 -- Si --> B115[Guardar coordenadas actuales con la codonría (x, y, orientación)]
    B114 -- No --> B116{¿los sensores infrarrojos detectan un obstáculo a 1 metro?}
    B115 --> B116
    B116 -- Si --> B117[Recorner perímetro del MPS]
    B116 -- No --> B118{¿la distancia entre las coordenadas actuales y el cuadrante objetivo es menor a 0.1 metros?}
    B118 -- Si --> B119[Calcular distancia con el laser]
    B118 -- No --> B120[Orar respecto a su ID 2]
    B119 --> B120
    B120 --> B121[Procesar eventos]
    B121 --> B122[Tomar fotografía]
    B122 --> B123{¿se identificó el giro correcto de las cámaras?}
    B123 -- Si --> B124[Calcular distancia con el laser]
    B123 -- No --> B125[Calcular distancia con el laser]
    B124 --> B126[Orar respecto a su ID 2]
    B125 --> B126
    B126 --> B127[Marcar con "x" el cuadrante objetivo en la cuadrícula]
    B127 --> B128{¿las coordenadas actuales son iguales a las coordenadas del cuadrante objetivo?}
    B128 -- Si --> B129[Guardar coordenadas actuales con la codonría (x, y, orientación)]
    B128 -- No --> B130{¿los sensores infrarrojos detectan un obstáculo a 1 metro?}
    B129 --> B130
    B130 -- Si --> B131[Recorner perímetro del MPS]
    B130 -- No --> B132{¿la distancia entre las coordenadas actuales y el cuadrante objetivo es menor a 0.1 metros?}
    B132 -- Si --> B133[Calcular distancia con el laser]
    B132 -- No --> B134[Orar respecto a su ID 2]
    B133 --> B134
    B134 --> B135[Procesar eventos]
    B135 --> B13
```

37

Momento 3. Búsqueda marcador ArUco

Luego de que se haya encontrado la caja, es necesario que el robotino recorra la caja con la cámara mirando hacia ella todo el tiempo, para poder tomarle foto al marcador ArUco. Esto con el propósito de identificar qué estación es aquella a la cual fue, para luego así almacenar su posición y código al que pertenece la “MPS”.

La forma en la cual el Robotino realiza esta operación es mediante señales de voltaje, que por proximidad mandará un valor exacto al cual se alejará o acercará manteniendo una distancia de 10 centímetros dependiendo del caso para seguir la casa, las opciones que se le da para mantenerse cerca de la caja son las siguientes

- ESCAPE_V = 2.0f;
- WALL_LOST_V = 1.9f;
- WALL_FOUND_V = 1.1f;
- WALL_FOLLOW_V = 1.2f;
- NEW_WALL_FOUND_V = 2.0f;

La lectura que se hacen de voltajes por medio de los infrarrojos corresponde a las lecturas de los sensores 0,1,8 de tal manera que el Robotino siempre esté mirando a la caja para realizar la captura del marcador Ar. Los sensores sobrantes recibirán señales de voltajes un poco más bajas de tal forma que el robotino mediante la función del *omnidrive()* *setvelocity()* siempre mire de frente, es decir rotar a una velocidad 0,34 rad/s aproximadamente de a 20 grados, para que la cámara permanezca perpendicular a la pared o caja que esté frente a ella, y así tomar las fotografías necesarias.

Después de entrar al caso correspondiente, al encontrar la caja a 10 cm a un voltaje de 1.1 V, realizará el seguimiento de manera horaria o antihorario según corresponda con un nuevo voltaje de 1.2 V para mantener la distancia estable entre el robot y la estación que halló, donde se le da el valor del sensor que lo detectó, la dirección y rotación para seguir la caja, por ejemplo: *rotate(escapeVector[curWallSensor], dir, rec::robotino::api2::deg2rad(followAngle));*

El Robotino además de ello al seguir la caja, da el valor de velocidad a la cual se debe mover según la distancia a la que se encuentre del objeto hallado, los valores según el caso son los siguientes:

- SLOW_VELOCITY 0.05f (en m/s)
- MEDIUM_VELOCITY 0.1f en (m/s)

- VELOCITY 0.12f (en m/s)
- FAST_VELOCITY 0.14f (en m/s)
- ANGULARVELOCITY 0.5f (en deg/s)

Así, durante las lecturas de los sensores infrarrojos en la búsqueda de la caja permanecerán estables y mantendrán una distancia prudente.

Como se mencionó anteriormente, el robotino puede rodear la caja en sentido horario o antihorario, esto va a depender de qué dirección venga el robot, y a que fila o cuadrante se dirige. Esto se hace ya que, en el espacio de trabajo, si hay una caja cercana a una pared, es mejor que rote en el sentido en el que los sensores no detectan la pared. De esta manera, la orientación predeterminada en la que se hace el rodeo de la cámara es horaria, sin embargo, existen 2 circunstancias cuando la va a rodear en sentido antihorario. La primera condición es si el cuadrante en el que está la caja se encuentra en la columna 3 de la matriz de la matrícula *grid*, y si la dirección que toma el robotino es suroriental. La segunda, es si el cuadrante con la caja se encuentra en la fila 9 del *grid*, y si el robotino va en dirección suroccidente.

El proceso de rodear la caja va a ocurrir hasta que se identifique un código ArUco en las fotos tomadas por la cámara. Una vez obtenida la información del marcador necesaria, el ID y el vector de rotación, se procede a imprimir las coordenadas del marcador y la orientación de este. Con el láser se mide la distancia del robotino al ArUco, y con las posiciones del robot dadas por la odometría, se calculan las coordenadas en el marco de referencia global del proyecto, donde se encuentra el marcador ArUco. También se halla la orientación del marcador utilizando el vector de rotación y la odometría, y por último se imprime en que cuadrante se encuentra y las coordenadas de este. Una vez el robotino haya identificado todos los marcadores, se va a imprimir la matriz de la cuadrícula, marcado con los 3 en los cuadrantes con caja, y el tiempo que se demoró el robotino en realizar toda la búsqueda. Un ejemplo de lo que se imprime se puede ver a continuación:

```
-----CAJA ENCONTRADA -----
-> ID: 26=(292.412,156.401) (408.858,163.59) (407.089,343.387) (299.102,303.189)
-> CUADRANT--> X: 3, Y: 0
-> COORDINATES CUADRANT--> X: 4.2, Y: 2.1
-> COORDINATES QR--> X: 4.1764, Y: 2.33907
-> ROTATION QR: 52.54
```

Figura 25. Ventana de comando: ID, Cuadrante, orientación, coordenadas cuadrante y QR de una estación

```

1111
1111
1111
3111
1111
1113
1111
1111
1111
1111
3111
Tempo total: 161 segundos

```

Figura 26. Escenario con estaciones registradas en columna 0 fila 3, columna 0 fila 9 y columna 3 fila 6

Búsqueda de cajas no identificadas en el primer paso.

Después de que se revisaron todos los cuadrantes donde se calculó que había una caja por el primer paso, puede que no se haya logrado identificar los 3 marcadores ArUco. Ya sea porque desde el primer momento no se marcó el 3, que puede ocurrir por varias razones, pero la principal, es porque desde el punto de vista del robotino, había una caja detrás de la otra, haciendo imposible que el láser lograra identificar la caja que estaba escondida. Otra razón por la que después de revisar cada cuadrante no se encontraron todos los ArUco, es debido a que se estimó que había una caja en un cuadrante donde, no solo no había una caja ahí, sino que tampoco estaba en los vecinos.

En estos casos es necesario encontrar las cajas que faltan, para esto se hizo que el robotino, después de encontrar el último ArUco del primer paso, se voltee y empiece rodear la caja de espaldas de la misma manera que se rodeaba en el paso anterior, pero esta vez, en lugar de hacer que el sensor infrarrojo 0 sea el que esté orientado hacia la caja, va a ser el sensor número 4, el cual se encuentra en la parte trasera del Robotino. Esto se hace para que la cámara pueda revisar todo el espacio para buscar las cajas faltantes, sin tener en cuenta la última caja que ya se identificó. Mientras rodea la caja, el Robotino va a estar tomando fotos y tomando medidas del láser de distancia. Cuando se identifique el color naranja se va a decir que se identificó una caja, pero la manera para saber si ya fue procesada, es utilizando las distancias del láser, que de igual manera que en el primer paso, se va a hallar el cuadrante en el que se encuentra la caja usando el promedio de los valores del láser, y la posición y orientación del robot. Ya conociendo en qué cuadrante se encuentra la caja, se busca si este ya está marcado con un 3 en el *grid*, y si ya está procesado. Si eso es verdad, el robotino continúa recorriendo la caja de espaldas, pero si en el *grid* el cuadrante está marcado con un n° 1, se dice que es una caja nueva. Cuando pasa esto el robot para teniendo la caja de frente a la cámara, y se adelanta hacia la caja a una velocidad de 0.13 m/s. Una vez algún sensor infrarrojo detecta la caja, este empieza a rodearla hasta detectar el marcador ArUco, y va a guardar su ID, coordenadas y orientación,

y también va a marcar el 3 en el *grid* en el cuadrante donde encontró la caja. Si al finalizar este proceso todavía faltan cajas por descubrir, se repite, si ya se encontraron las 3, el programa se detiene.

Conclusiones:

En este proyecto se ha hecho uso de diferentes técnicas para el desarrollo y buena ejecución de lo solicitado en la asignatura diseño mecánico en énfasis y por la primera fase de la competencia Robocup.

Se evidencia el cumplimiento en su totalidad de tanto los objetivos específicos como el general, ya que primero, el robotino es capaz de identificar las estaciones presentes en el lugar de trabajo e identifica los códigos fijados a las cajas (MPS) y de igual forma presenta en la ventana de comandos las orientaciones y posiciones de las estaciones y de igual manera de los AruCos.

La velocidad con la que encuentra las estaciones y se traslada hasta las estaciones de igual manera es pertinente para el escenario con el que se está trabajando. Ya que no se evidencian colisiones que comprometan la integridad de las MPS o del Robotino, y en cuanto a los códigos, los detecta al poco tiempo de haberse posicionado en frente de las estaciones.

Se evidencia un buen procesamiento a las imágenes que toma la cámara, ya que el robotino determina la ubicación de las MPS gracias a su color y tamaño.

Por otra parte, el hecho de realizar varias pruebas con distintos escenarios con posicionamientos de las MPS nos permite ver el grado de confiabilidad que tiene el código desarrollado ya que en la mayoría de los casos no se presentan colisiones, se ejecutan de manera correcta los casos particulares (por ejemplo: el detectar solo 2 estaciones de 3), el tiempo de recorrido no supera los 3 minutos, lo cual es ideal para esta etapa de la competencia ya que es el tiempo que tienen los participantes para mapear el área donde están las estaciones.

Con este documento se espera que los futuros integrantes que quieran seguir con el proyecto para participar en la Robocup Logistics League, puedan adquirir y aumentar sus conocimientos sobre los temas tratados aquí y tengan una introducción más accesible a la competencia y al funcionamiento del Robotino.

Referencias:

- [1] Comité Técnico de Competición ICAPS 2017, «Concurso de Planificación y Ejecución para Logística Robots en Simulación – Reglas y Regulaciones – ICAPS 2017. tecnología reps.,» de Conferencia Internacional sobre Planificación y Programación Automatizadas, 2017.
- [2] C. Estrebou, «ALGORITMOS DE IDENTIFICACIÓN DE PIEL HUMANA Y SU RELACIÓN CON LOS SISTEMAS DE COLOR. SU APLICACIÓN A LA SEGMENTACIÓN DE PIEL BASADA EN PÍXELES,» Facultad de Informática - Universidad Nacional de La Plata, La Plata, 2020.
- [3] Open Source Computer Vision, «Image Thresholding,» OpenCV, [En línea]. Available: https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html. [Último acceso: 25 de mayo 2022].
- [4] F. J. Romero-Ramirez, R. Muñoz-Salinas y R. Medina-Carnicer, «Speeded up detection of squared fiducial markers,» Image and Vision Computing, vol. 76, pp. 38-47, 2018.
- [5] S. Garrido-Jurado, R. M. Salinas, F. Madrid-Cuevas y R. Medina-Carnicer, «Generation of fiducial marker dictionaries using mixed integer linear programming,» Pattern Recognition, n° 51, pp. 481-491, 2016.
- [6] Intel RealSense Technology, «Intel® RealSense™ D400 Series Product Family Datasheet,» Junio 2019. [En línea]. Available: <https://ip.festo-didactic.com/InfoPortal/Robotino/document/realsense.pdf>. [Último acceso: 25 Mayo 2022].
- [7] OPTO-ELECTRONIC DEVICES DIVISION ELECTRONIC COMPONENTS GROUP SHARP CORPORATION, «Device Specification for Analog Output Type Distance Measuring Sensor Model No. GP2Y0A41SK0F,» 2005.
- [8] "Speeded up detection of squared fiducial markers", Francisco J. Romero-Ramirez, Rafael Muñoz-Salinas, Rafael Medina-Carnicer, Image and Vision Computing, vol 76, pages 38-47, year 2018
- [9] "Generation of fiducial marker dictionaries using mixed integer linear programming", S. Garrido-Jurado, R. Muñoz Salinas, F.J. Madrid-Cuevas, R. Medina-Carnicer, Pattern Recognition: 51, 481-491, 2016
- [10] «Robotino 4,» FESTO, [En línea]. Available: <https://ip.festo-didactic.com/InfoPortal/Robotino/Overview/DE/index.html>. [Último acceso: 25 Mayo 2022]
- [11] «Main Page,» Robotino Wiki, 31 Mayo 2021. [En línea]. Available: https://wiki.openrobotino.org/index.php?title=Main_Page. [Último acceso: 25 Mayo 2022].

Anexos:

Anexo A: Clases propias

```
class cuadrant
{
public:
    int index_x;
    int index_y;
    double x_0;
    double x_f;
    double y_0;
    double y_f;
    int procesado = false;

};
```

Clase que define los cuadrantes de la matriz del espacio.

Parámetros:

Index_x	Fila del cuadrante en la matriz.
Index_y	Columna del cuadrante en la matriz.
X_0	Coordenada en x donde empieza el cuadrante en metros.
X_f	Coordenada en x donde finaliza el cuadrante en metros.
y_0	Coordenada en y donde empieza el cuadrante en metros.
y_f	Coordenada en y donde finaliza el cuadrante en metros.
Procesado	El cuadrante fue revisado por el robotino.

```
class distsensors
{
public:
    double orientation_x;
    double orientation_y;

};
```

Clase que define las orientaciones de los sensores infrarrojos con respecto al sistema de coordenadas global.

Parámetros:

Orientation_x	Coseno del ángulo correspondiente a cada sensor.
Orientation_y	Seno del ángulo correspondiente a cada sensor.

Anexo B: Clases API Robotino

Clase para la conexión del computador con el robotino. Casos de conexión. (Documentación API)

class MyCamera : public Camera

Clase de la cámara. Se ajustan los parámetros de la cámara y se guardan las imágenes. Se llaman cuando ocurre un evento cuando se llama la función `com.processEvents()`. (Documentación API)

Cuando ocurre el evento, recorta la imagen (**`crop(string& source, Mat* cropImg)`**), hace la detección del color naranja (**`void colorDetection(Mat& img, int* naranja)`**), **`void colorDetectionContraluz(Mat& img, int* naranjaContraluz)`**, y la identificación de los marcadores ArUco. En caso de que se detecte el color naranja, variable global `int founBox = true`. En caso de que se encuentre un ArUco, variable global `foundAruco = true`.

class MyLaserRangeFinder : public rec::robotino::api2::LaserRangeFinder

Clase que define parámetros del láser en la cámara intel realsense d435.

Anexo C: Objetos

MyCom com;
MyLaserRangeFinder rangefinder;
MyCamera camera;
OmniDrive omniDrive;
DistanceSensor sensors[9];
Odometry position;
array<cuadrant, 40> cuadrantes;

```
array<array<int, 4>, 10> grid{
    { { { 1, 1, 1, 1 } },
      { { 1, 1, 1, 1 } },
      { { 1, 1, 1, 1 } },
      { { 1, 1, 1, 1 } },
      { { 1, 1, 1, 1 } },
      { { 1, 1, 1, 1 } },
      { { 1, 1, 1, 1 } },
      { { 1, 1, 1, 1 } },
      { { 1, 1, 1, 1 } },
      { { 1, 1, 1, 1 } },
      { { 1, 1, 1, 1 } },
    },
};
```

Iniciación de matriz de 4 x 10 llena de unos, que representan los cuadrantes de 70 cm en los cuales se dividió el espacio. 1 representa que el cuadrante está vacío. 3 representa que hay una caja en el cuadrante.

Anexo D: Funciones

Void map1()

Le asigna los parámetros de una clase cuadrant (index_x, index_y, x_0, x_f, y_0, y_f) a un arreglo de 40 cuadrantes, que representan los cuarenta cuadrantes del grid.

void checkCuadrant(double x, double y, int* index_x_out, int* index_y_out, cuadrant* cuadranteEstoy)

Lee las coordenadas x y y del robotino, y revisa en qué cuadrante se encuentra.

Parámetros:

X x actual del robotino en metros.

Y y actual del robotino en metros.

Index_x_out fila del cuadrante donde está el robotino.

Index_y_out columna del cuadrante donde está el robotino.

cuadranteEstoy Cuadrante donde se encuentra el robotino.

void checkBoxCuadrant(float voltage, int index, double x, double y)

Encuentra las coordenadas x y y, del objeto que detectó el sensor infrarrojo con mayor voltaje. Encuentra el cuadrante que corresponde a dichas coordenadas, y le asigna un 3 en el grid.

// Relación entre voltaje y distancia

// Distancia = $12.154 * (\text{voltaje}^{1.074})$

Parámetros:

Voltage voltaje del sensor con mayor voltaje.

Index Número del sensor con mayor voltaje.

X x actual del robotino en metros.

Y y actual del robotino en metros.

void eraseBox(double x, double y)

Borra el 3 del grid del cuadrante en el que se encuentra el robotino.

Parámetros:

X x actual del robotino en metros.
Y y actual del robotino en metros.

int crop(string& source, Mat* cropImg)

Recorta las fotos del robotino.

Parámetros:

Source Dirección de la imagen.

cropImg Imagen recortada.

void colorDetection(Mat& img, int* naranja)

Detecta el color naranja de la imagen recortada. Se ajustan los parámetros de hue, saturación y valor mínimo y máximo para el color naranja. Se cuentan los pixeles blancos. Si son más de mil, detectó el naranja de las cajas.

Parámetros:

Img Imagen que se va a procesar.

Naranja Identificó el color naranja en la imagen.

void colorDetectionContraluz(Mat& img, int* naranjaContraluz)

Misma función del colorDetection, pero para cajas a contraluz (Diferente color, por lo tanto, diferentes parámetros de hue, saturación y valor)

Parámetros:

Img Imagen que se va a procesar.

Naranja Identificó el color naranja en la imagen.

void arucoReader(string path, int* sizeMarker)

Detecta el marcador ArUco y lee su ID.

Parámetros:

path Dirección de la imagen.

sizeMarker Número de marcadores ARucos detectados.

void outlier(list<double> distancias, double* distanciaPromedioOut)

Encuentra los valores atípicos de una lista con datos, se borran de la lista, y halla el promedio de los valores que no son valores atípicos.

Primero se ordena la lista de menor a mayor, luego se hallan las medias del cuartil 1 y del cuartil 3. La resta de estas se denomina rango intercuartílico (IQR). Y si el valor mayor de la lista es mayor a 1.5 veces el IQR, se considera un valor atípico y se borra de la lista. Se hace lo mismo para el valor más pequeño. Luego, con la lista final, se halla el promedio.

void init(const std::string& hostname)

Inicia los actores. Crea un arreglo para los 9 sensores infrarrojos, inicia la cámara y la conexión.

Parametros:

Hostname Puerto del robotino (192.168.0.104)

void checkSensors(int* indexMax)

Recorre el arreglo de los sensores infrarrojos para encontrar que sensor tiene el mayor voltaje, y cuál es su valor. Si el voltaje máximo es mayor a 1,0 voltios, nearBox = true (Se encuentra cerca de una caja)

Parámetros:

indexMax Número del sensor con mayor voltaje.

void setVelocityDirect(double dist_x, double dist_y, double* x_vel, double* y_vel)

Asigna una velocidad en x y en y dependiendo de la distancia. Las distancias en x y y se dividen entre el mismo factor para llegar a una velocidad alta, que no pase de 0.4 m/s, para que no ocurran deslizamientos en las ruedas y hallan cambios de dirección. Si la distancia en x es mayor a 5 metros, se dividen entre 18. Si está entre 3 y 5 metros, se dividen entre 12. Y si son menores a 3 metros, se dividen entre 8.

Parámetros:

Dist_x Distancia en x del robotino a la posición a la que debe ir en metros.

Dist_y Distancia en y del robotino a la posición a la que debe ir en metros.

x_vel Velocidad en x en metros por segundo.

Y_vel Velocidad en y en metros por segundo.

void goToQuadrantDirect(quadrant cuadrante_obj, double ang_actual, double x_actual, double y_actual, int* side)

Lleva al robotino a la posición deseada.

Primero encuentra las coordenadas en x y y a las que tiene que ir, y resta las coordenadas objetivo con las actuales del robotino para hallar la distancia que tiene que recorrer en x y en y. Con estos valores se hallan las velocidades en x y en y con la función `setVelocity()`. Luego, gira a 0.2 rad/s sobre su propio eje hasta que quede en la misma orientación en la que comenzó, utilizando la función de `setVelocity()` del objeto `omniDrive()`. Gira en sentido horario o antihorario dependiendo de cuál es el camino más corto. Dependiendo de los signos de las distancias a recorrer en x y y (si son positivos o negativos), se sabe si el robotino debe ir en dirección noroccidental, suroccidental, suroriental, o nororiental. Dependiendo de la dirección que debe tomar el robotino, se revisan si los sensores que se encuentran entre el robotino y su destino sobrepasan los 1.0 voltios, si lo hacen se considera que la caja se encuentra en el camino, y el robotino hace maniobras para esquivar la caja, moviéndose en x y y, hasta que el camino queda despejado. Después de esquivar la caja, se hallan las nuevas distancias, y las nuevas velocidades para la posición en la que quedó el robotino.

Por último, se vuelve a estimar la dirección en la que va el robotino dependiendo si la distancia en x y y son positivos o negativos. De acuerdo a esto se le pone velocidad al robotino con `omniDrive.setVelocity()`, con las velocidades halladas previamente, y se utiliza la función de `distances`, para un objeto `positions` de la clase `Odometry`, (`positions.readings(x, y, phi)`), para hallar las coordenadas de x, y y, y la orientación del robotino. A estos se les resta la medición inicial que se hace al comienzo de la operación para tener los resultados en 0 desde el principio. Esta acción se realiza dentro de un ciclo `while`, que funciona si no ha llegado a la posición deseada, y si `nearBox = false`, que significa que los sensores infrarrojos no han detectado nada. Dentro del `while` se pone la condición, que, si el robot está a menos de 1 metro de distancia de la meta, se disminuye la velocidad dividiendo la velocidad en x y en y por 1.5, y se llama la función de `checkSensors()` para que empiece a revisar los sensores, a ver si se encontró la jaca y parar.

Parámetros:

Cuadrante_obj	Cuadrante al que debe ir el robotino.
Ang_actual	Ángulo actual del robotino en radianes.
X_actual	x actual del robotino en metros.
Y_actua	y actual del robotino en metros.
int* side	Dirección que debe tomar el robotino.

void closestCuadrant(cuadrant initial, list<cuadrant> cuadrante2goClose, cuadrant* cuad2go)

Encuentra que cuadrante del grid marcado con 3, es el que se encuentra más cerca de la posición actual del robotino.

Parámetros:

Initial	Cuadrante en el que se encuentra el robotino.
cuadrante2goClose	Lista de los cuadrantes que están marcados con un 3 en el grid.
Cuad2go	Cuadrante más cercano. Cuadrante al que se tiene que ir.

void checkBoxLaser(double phi_l, double x_l, double y_l)

Usa las distancias actuales del robotino, su orientación, y la distancia de la caja hallada con la función `findDistanceAndCuadrantLasr()`, para verificar en que cuadrante se encuentra la caja, y marcar dicho cuadrante con un 3 en el grid.

Parámetros:

Phi_l **Ángulo actual del robotino en radianes.**
X_l x actual del robotino en metros.
Y_l y actual del robotino en metros.

void findDistanceAndCuadrantLasr()

Esta función halla la distancia del robotino a la caja medido con el láser de la cámara. Primero, el robotino empezará a rotar sobre su eje, y comenzará a tomar los valores de distancia que envía el láser, cuando estos valores sean mayores a 0 y menores a 6.5 metros, que es la distancia máxima del espacio, y la cámara detecte el color naranja, se dirá que se detectó una caja, y se empezarán a guardar los valores del láser en una lista siempre y cuando no sobrepasen el límite de la zona expuesta (estos van a depender de la rotación del robotino). Se van a parar de tomar si el láser detecta un 0, si la medida más reciente es 1 metro mayor o menor que la anterior, o si deja de detectar el color naranja. A la lista final se le borran los valores atípicos y se halla el promedio de las distancias con la función de `outlier()`. También se halla el promedio de los ángulos de cuando se encontró la caja y de cuando la perdió. Estos se introducen a la función de `checkBoxLaser()` para determinar en qué cuadrante está la caja.

void findDistanceAndCuadrantLasr2(int* cajaValida)

Esta funciona del mismo modo que `findDistanceAndCuadrantLasr()`, pero se utiliza cuando no se detectan las 3 cajas, y esta, mientras busca las cajas faltantes durante la función `wallFollowBack()`, verifica que la caja que se encuentre no sea una que ya está procesada. Por lo tanto, revisa el cuadrante de la caja, y si no ha sido procesada es una nueva caja, `cajaValida = true`.

Parámetros:

cajaValida La caja no ha sido procesada, y es válida para ir a procesar.

void checkNeighbours(cuadrant* cuadrantVecino)

Esta función sirve para buscar si hay cajas en los cuadrantes vecinos, del cuadrante en el que se encuentra el robotino.

Primero el robotino gira sobre su propio eje

void rotate(const double* in, double* out, double rad)

Esta función permite al robot girar de manera horaria alrededor de un objeto

void rotateInPlace(double* v, double rad)

Esta función permite que el robotino pueda rotar en su lugar sin tener que desplazarse

void addScaledVector(double* srcDest, const double* uv, double scale)

Añade un Vector, de tal manera que los destinos y las medidas de los voltajes según los sensores puedan escalar según la medida del voltaje, es decir agregar y sustraer voltaje de ellos para seguir un objeto.

void normalizeVector(double* v)

Para cada voltaje extraído de los sensores normaliza los valores de los voltajes percibidos mediante Pitágoras, así los valores de estos son un valor más estable y único por medición.

float getNextNeighbourDistance(int index)

Devuelve el valor del sensor más próximo con un voltaje adecuado para realizar el seguimiento del objeto.

float getPrevNeighbourDistance(int index)

Devuelve el valor del sensor anterior con el que tomo la medida para hacer retroalimentación de los valores de voltaje encontrados por los sensores.

WallFollow()

Asigna primero en `escapeVector[9][2]` un arreglo donde los valores de los voltajes de cada sensor del 0 al 8 donde la primera columna son los sensores y la segunda son los valores de los voltajes de cada uno, luego le entrega un valor de rotación a cada uno de los sensores, ahora para que el robotino realice el seguimiento de la pared se determinan variables para ello; Así mismo, se le asigna valores de velocidades para este realizar el desplazamiento y la rotación. Dependiendo del caso la función recorrerá cada uno de los sensores dándoles un valor a cada uno y luego asignándole un voltaje para determinar cuál sensor está recibiendo la señal, en caso de que 3 sensores estén recibiendo voltajes al mismo tiempo el robot intentara dejar de seguir el objeto. La única manera que el robot determine si tiene un objeto delante para seguir es mediante los sensores 0,1,8, los otros sensores servirán para redirigir la parte delantera (donde este la cámara del robotino) para seguirla de frente. Por último, revisa los sensores para en algún caso cambiar de pared por los sensores o indicar si está saliendo del objeto, de la misma forma mantiene los valores de los voltajes estables retroalimentándolos a las funciones de **getNextNeighbourDistance** y **getPrevNeighbourDistance** para poder seguir una dirección y una rotación, mediante las funciones que representan los cuadrantes mientras que realiza este proceso sabe en qué lugar del escenario esta.

WallFollowBack()

El principio de funcionamiento es igual que el que se determina con la función **WallFollow()** para seguimiento de objetos. Esta función según el caso se activará de tal manera que el seguimiento del objeto se haga de espaldas (lado contrario a la cámara) donde los sensores 4,5,6 son los que determinaran el seguimiento del objeto, así como los demás sensores servirán para que el robotino gire o se mueva y rote entorno a estos sensores.

wallFollowR()

Entendiendo ya el funcionamiento de la función **WallFollow()** se realizan ajustes a funciones correspondientes y nuevas como: **void rotateR(const double* in, double* out, double rad), rotateInPlaceR(double* v, double rad), void addScaledVectorR(double* srcDest, const double* uv, double scale), void normalizeVectorR(double* v), float getNextNeighbourDistanceR(int index), float getPrevNeighbourDistanceR(int index)**, de tal manera que este seguimiento con los mismos lineamientos anteriormente explicados pueda replicar el comportamiento pero en sentido antihorario. Por ejemplo, la variable **followangle** se realizará de manera negativa (es decir su valor con signo contrario) para que este realice su función de manera antihorario.

Anexo E: Instalación dependencias y librerías para correr el programa en C++

Para continuar con el proyecto se debe de necesitar los siguientes paquetes instalados en el ordenador o equipo de trabajo.

1. VisualStudio
2. Cmake
3. OpenCv
4. ArUco library
5. RobotinoApi

1. Visual Studio

- a. Diríjase a la página web donde se puede descargar el IDE para c++ <https://visualstudio.microsoft.com/es/> una vez allí de click en el botón de descargar

2. Cmake

- a. Diríjase a la pagina de Cmake <https://cmake.org/> en la parte superior derecha verá la pestaña de descarga haga click en ella
- b. Haga click en la distribución de binarios e instalación del programa

Binary distributions:

Platform	Files
Windows x64 Installer	cmake-3.23.1-windows-x86_64.msi

3. OpenCV

- Diríjase a la página de Opencv <https://opencv.org/releases/> una vez allí haga click en la opción de Windows, lo redirigirá a la página de source donde iniciara la descarga de opencv
- Puede familiarizarse con el programa siguiendo los siguientes tutoriales y seguir la guía de instalación también <https://www.youtube.com/watch?v=2FYm3GOonhk>

4. ArUco Library

- Diríjase a la página SourceForge.net y descargue la librería para ++ allí <https://sourceforge.net/projects/aruco/>
- Descargue la librería dentro de Opencv como por ejemplo en esta ruta: “C:\opencv\build\include\opencv2”

5. Robotino API

- Diríjase a la página robotino wiki https://wiki.openrobotino.org/index.php?title=Main_Page
- Una vez allí querrá hacer click en el botón de descargas. Luego descargue el api para la versión de Windows con la que opera su dispositivo.

API, Plugins & Packages

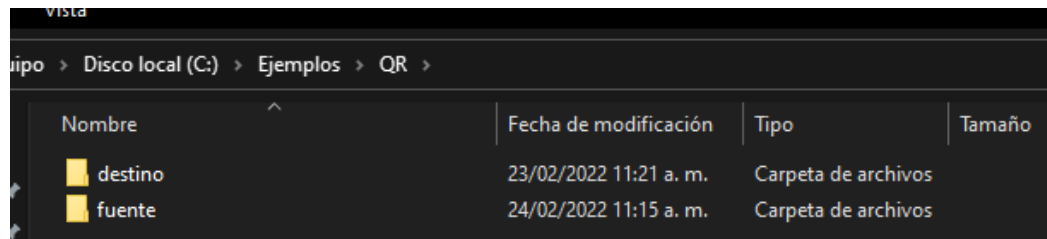
 API2

Version	OS	Compiler	Link
latest	Windows 32bit	Visual Studio 2010 32bit	download
latest	Windows 64bit	Visual Studio 2010 64bit	download

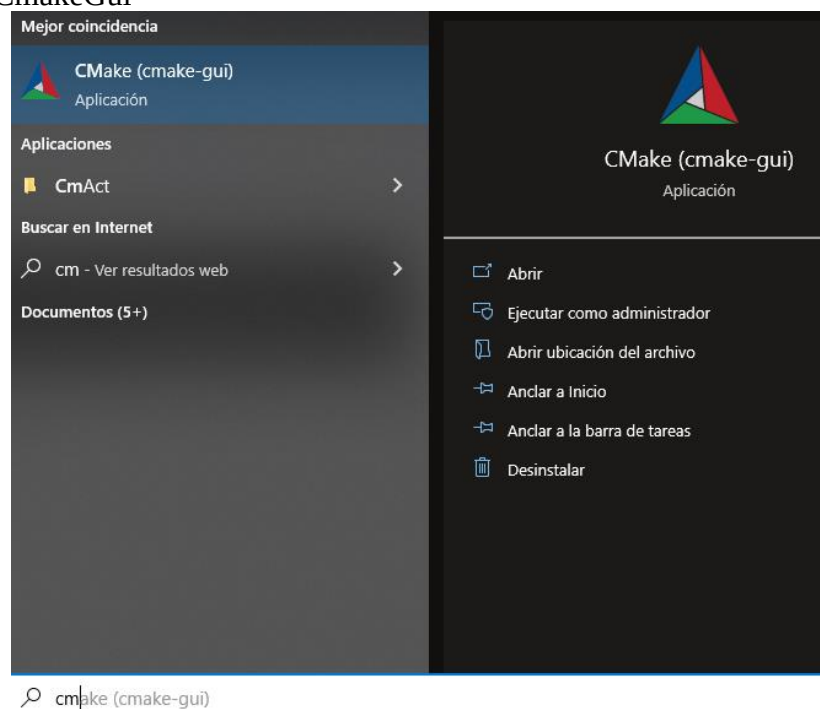
- Dentro de la siguiente ruta e.j “C:\Program Files\REC GmbH\robotino\api2\examples” Encuentra usted algunos ejemplos para familiarizarse con el lenguaje y el robotino.

6. Compilación de los programas Robotino Examples

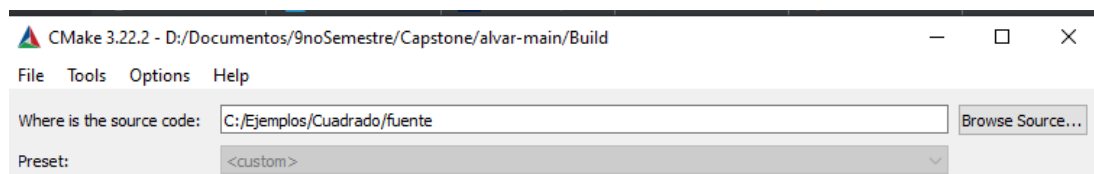
- Deberá crear en su dispositivo en la siguiente carpeta, por ejemplo: “C:\Ejemplos”
- Tendrá que crear dos carpetas dentro de ejemplos una donde este el source y otro donde este el realice



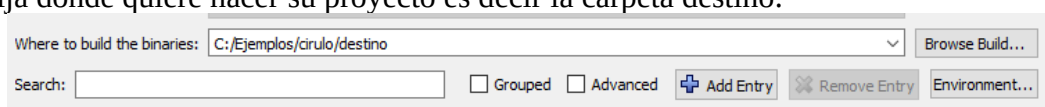
- c. Extraiga del siguiente directorio el ejemplo de circle por ejemplo: “C:\Program Files\REC GmbH\robotino\api2\examples\cpp\circle” copie y pegue Cmake y main.cpp a su carpeta de fuente
- d. Diríjase a CmakeGui



- e. Seleccione en fuente, la fuente del código por ejemplo:



- f. Elija donde quiere hacer su proyecto es decir la carpeta destino:

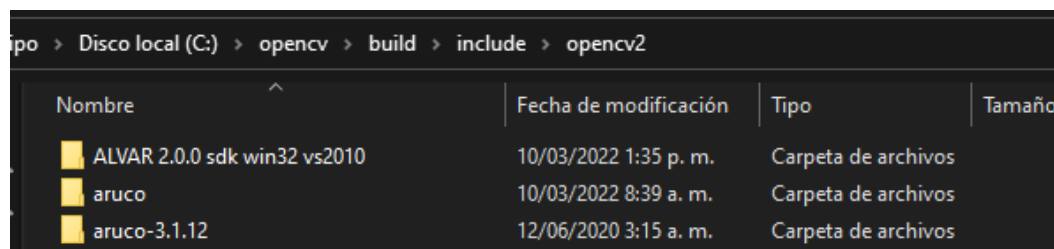


- g. Ahora dé a configurar y luego a generar.

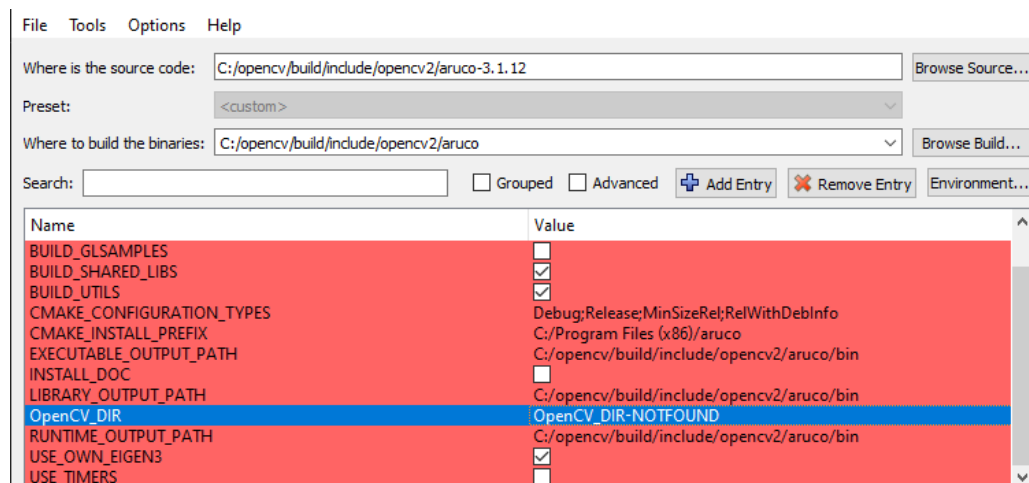
- h. Diríjase a la carpeta destino, por ejemplo: “C:\Ejemplos\CAMERA\destino” y abra el archivo .sln que ha creado
- i. Una vez allí compile el programa cambiando la dirección Ip para el robotino según sus necesidades
- j. Diríjase a la carpeta realice, por ejemplo: “C:\Ejemplos\CAMERA\destino\Release” presione la aplicación .exe y corra el ejemplo. En caso de que el programa le pregunte por algún .dll faltante diríjase por ejemplo “ C:\Program Files\REC GmbH\robotino\api2\bin” y copie y pegue en la carpeta release los .dll necesarios

1. Compilación ArUco Library.

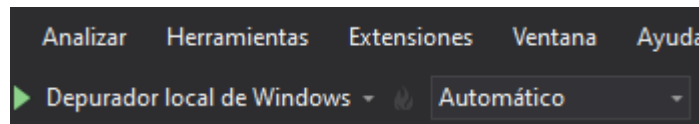
- a. Después de descargar las librerías de Aruco copie la carpeta dentro de opencv por ejemplo : “C:\opencv\build\include\opencv2”
- b. De la misma forma para utilizar Cmake realizaremos dos carpetas una será la fuente y otra el destino. Por ejemplo:



- c. Abrimos Cmake-gui
- d. En la parte de soruce elegimos aruco-3.1.12 y en Build elegimos la carpeta destino, por ejemplo: aruco
- e. Damos a configurar y luego generar, en caso de que no encuentre el “Path” a Opencv

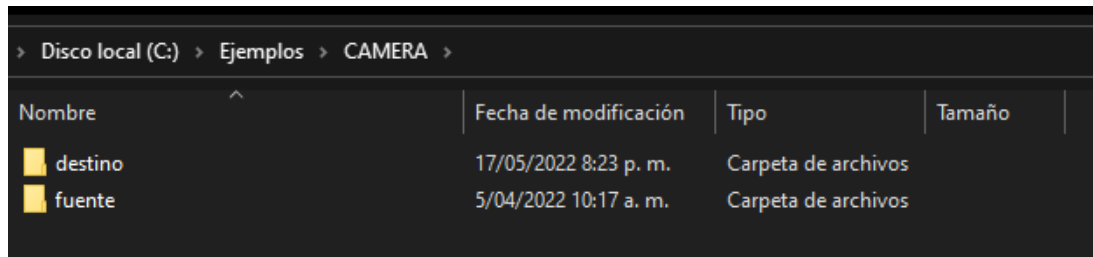


- f. Ingrese por ejemplo la siguiente dirección: “C:\opencv”. Una vez dentro vaya al .sln y luego dupere con el programa de Windows:

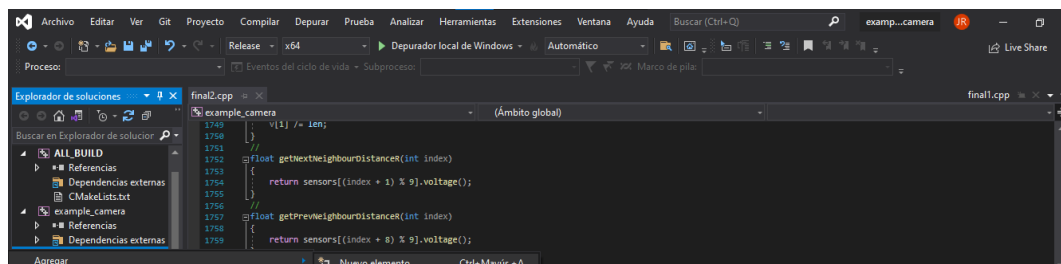
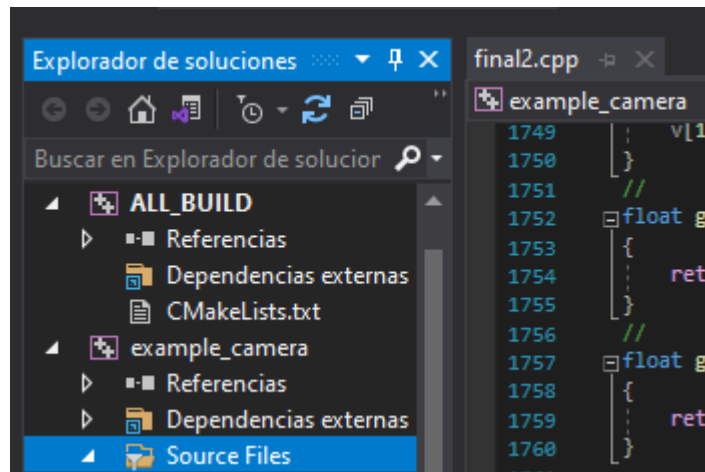


- g. Configuración Dependencias y librerías para el programa de la rcll

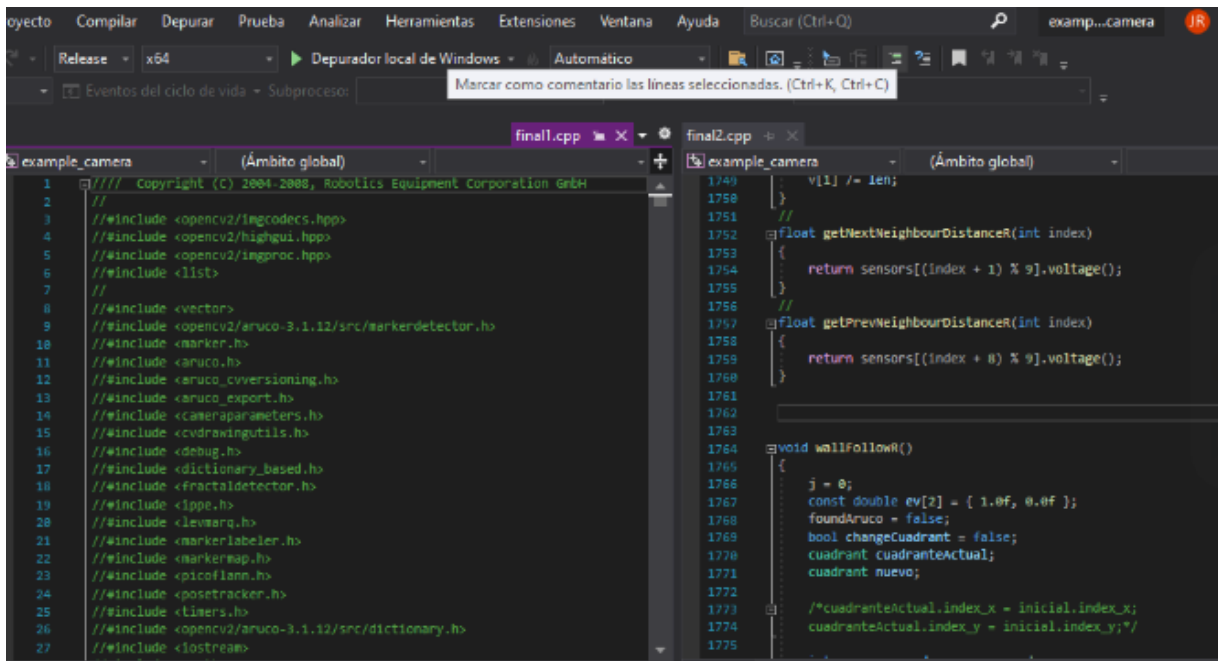
- h. Después de haber compilado por ejemplo el ejemplo de la cámara



- i. Con el proceso mencionado anteriormente tendrá que abrir el archivo .sln y de esta manera:
- j. Una vez dentro de VisualStudio, añadirá un código fuente con el código del programa rcll, en el ejemplo “final2” de la siguiente forma:



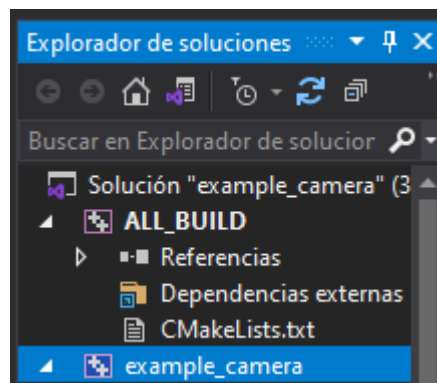
- k. Con click izquierdo en Source agregue un nuevo elemento tipo cpp. Luego en ese archivo nuevo ponga el código. Recuerde que para funcionar solo un archivo cpp debe estar activo para compilar, si tiene más archivos tendrá que comentarlos, como por ejemplo:



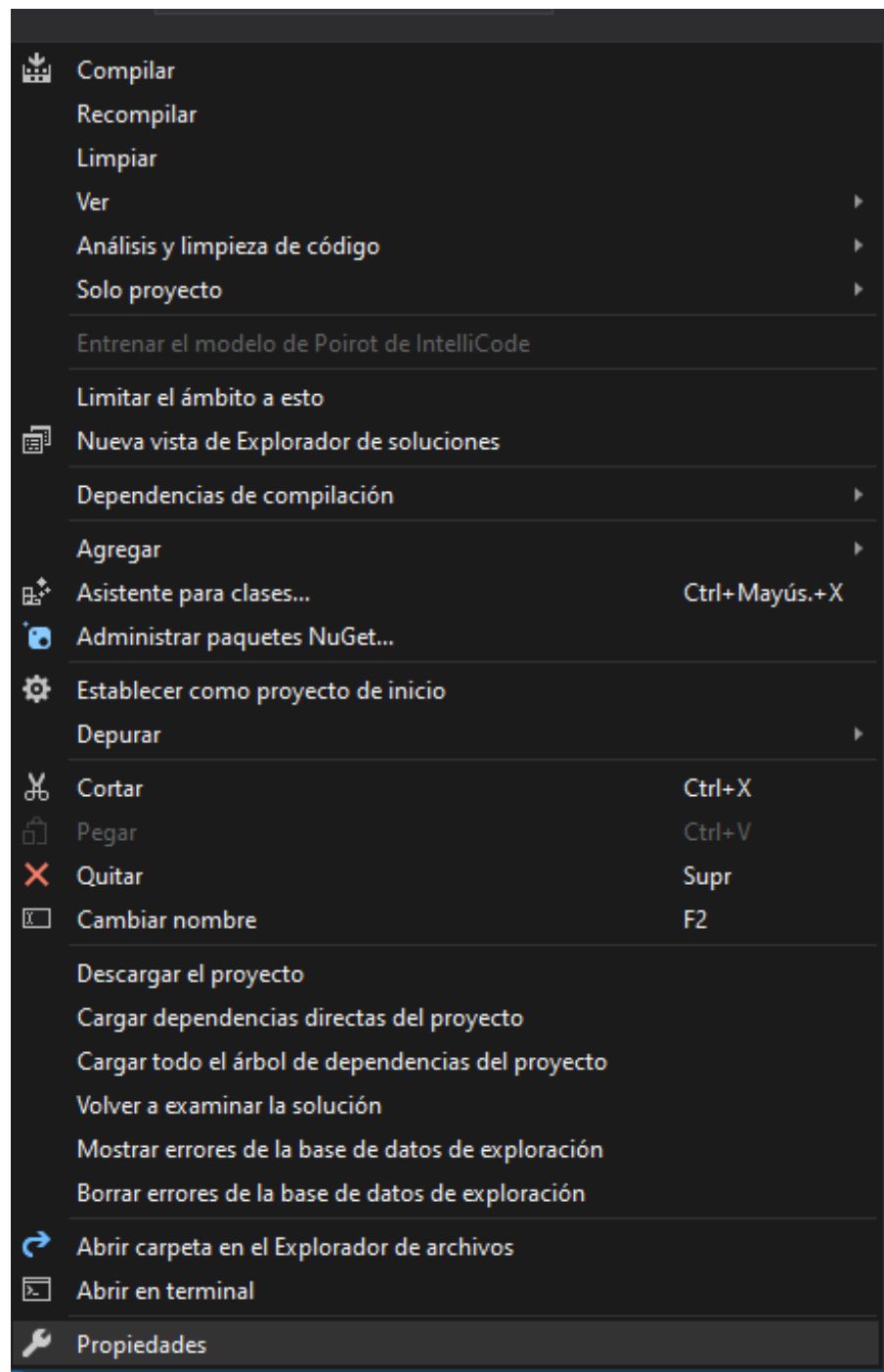
```
1 1 // Copyright (C) 2004-2008, Robotics Equipment Corporation GmbH
2 //
3 // #include <opencv2/imgcodecs.hpp>
4 // #include <opencv2/highgui.hpp>
5 // #include <opencv2/imgproc.hpp>
6 // #include <list>
7 //
8 // #include <vector>
9 // #include <opencv2/aruco-3.1.12/src/markerdetector.h>
10 // #include <marker.h>
11 // #include <aruco.h>
12 // #include <aruco_cvversioning.h>
13 // #include <aruco_export.h>
14 // #include <cameraparameters.h>
15 // #include <cvdrawingutils.h>
16 // #include <debug.h>
17 // #include <dictionary_based.h>
18 // #include <fractaldetector.h>
19 // #include <ippe.h>
20 // #include <levmarq.h>
21 // #include <markerlabeler.h>
22 // #include <markermap.h>
23 // #include <picoflann.h>
24 // #include <posetracker.h>
25 // #include <timers.h>
26 // #include <opencv2/aruco-3.1.12/src/dictionary.h>
27 // #include <iostream>
```

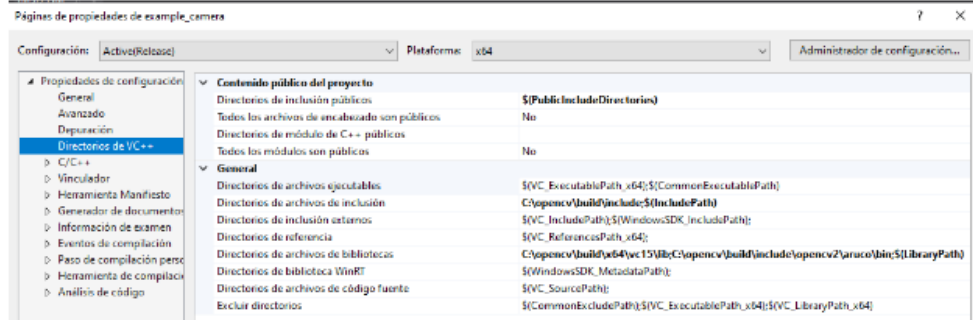
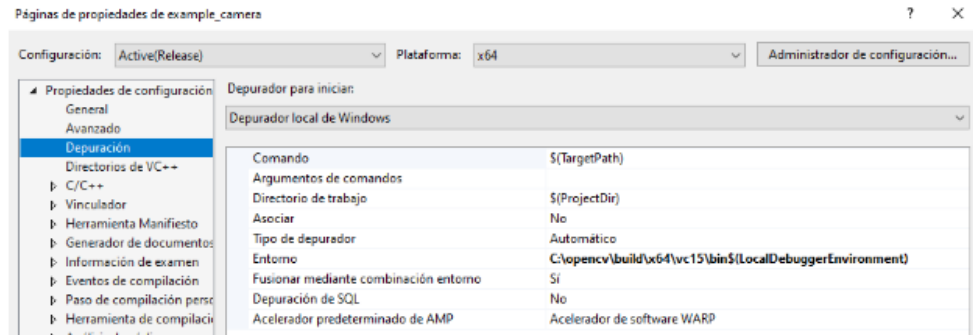
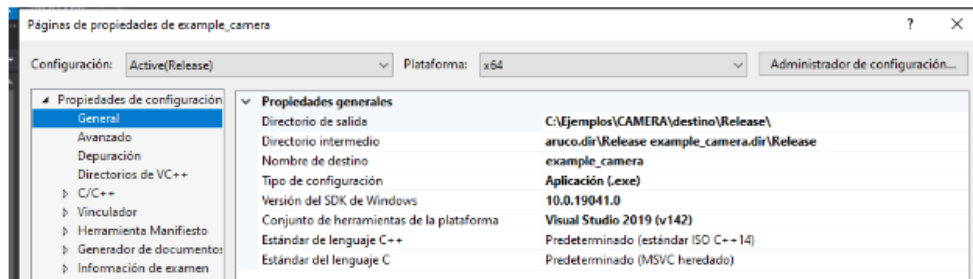
```
1749 v[1] /= len;
1750 }
1751 //
1752 float getNextNeighbourDistanceR(int index)
1753 {
1754     return sensors[(index + 1) % 9].voltage();
1755 }
1756 //
1757 float getPrevNeighbourDistanceR(int index)
1758 {
1759     return sensors[(index + 8) % 9].voltage();
1760 }
1761
1762
1763
1764 void wallFollowR()
1765 {
1766     j = 0;
1767     const double ev[2] = { 1.0f, 0.0f };
1768     foundAruco = false;
1769     bool changeCuadrant = false;
1770     cuadrant cuadranteActual;
1771     cuadrant nuevo;
1772
1773     /*cuadranteActual.index_x = inicial.index_x;
1774     cuadranteActual.index_y = inicial.index_y;*/
1775 }
```

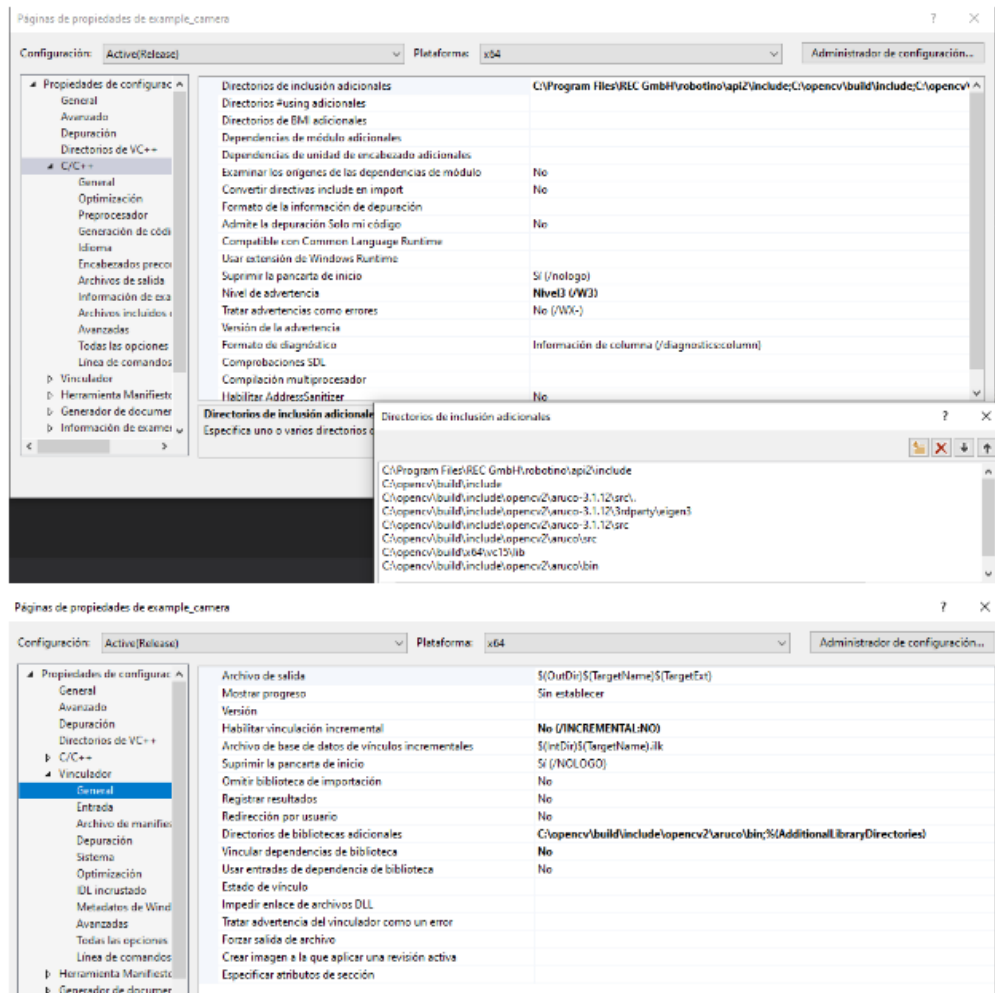
- l. Luego se configurarán lo recorrido para agregar las librerías siguiendo las propiedades del proyecto:



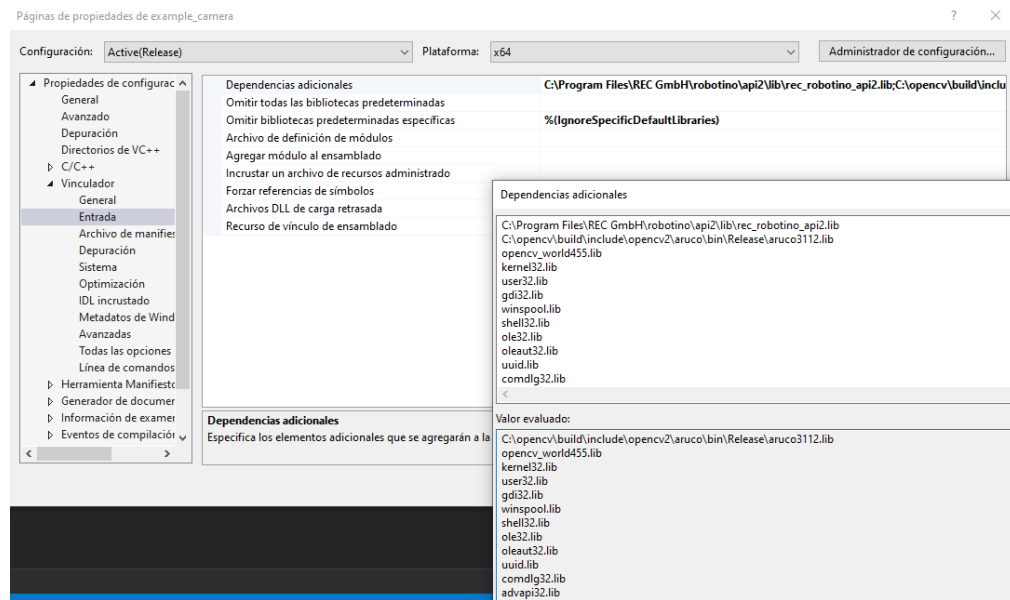
- m. Click derecho y dirigirse a propiedades.





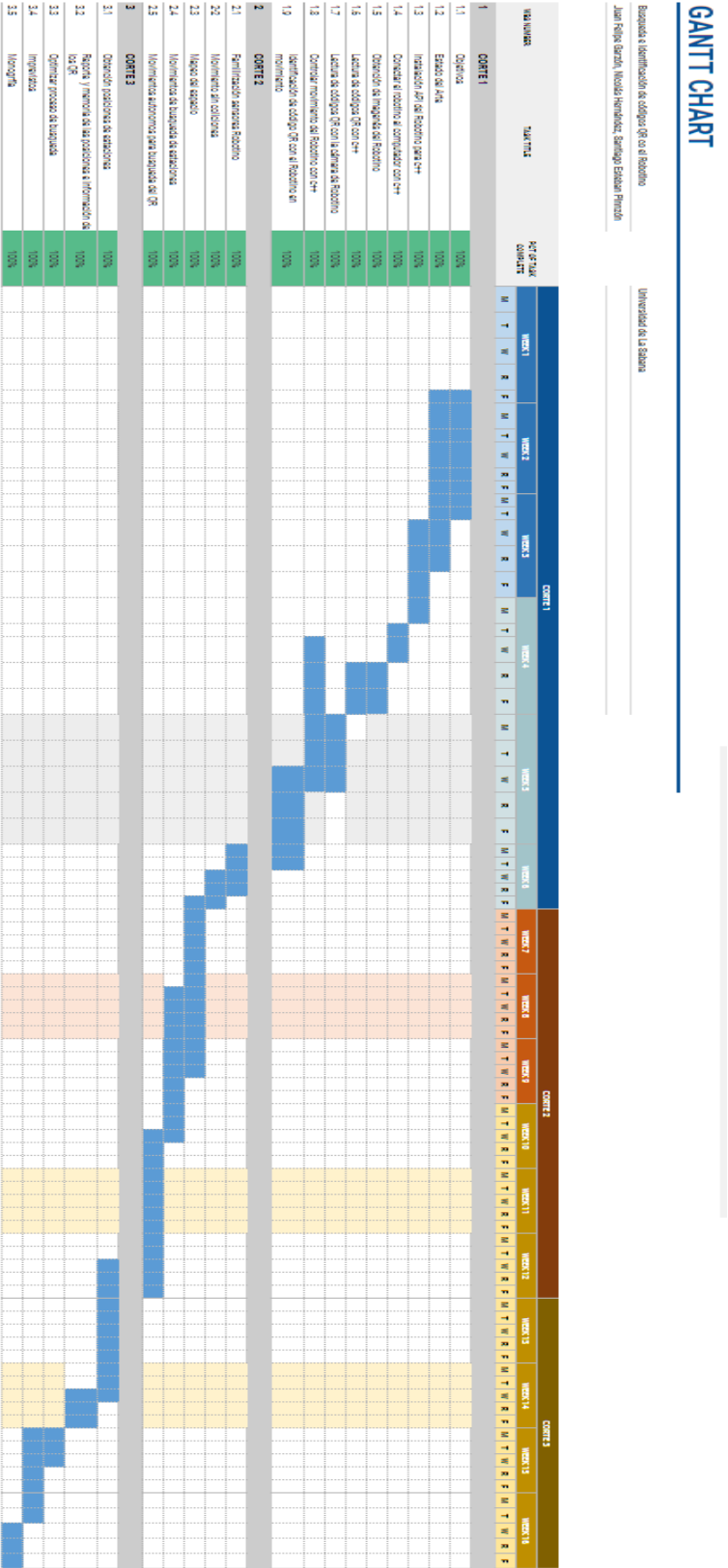


- n. En la entrada de los vinculadores incluya también advapi32.lib en la parte superior en dependencias adicionales



o. Luego se podrá compilar en release x64 utilizando ctrl+B

Anexo F: Diagrama de Gantt



1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100